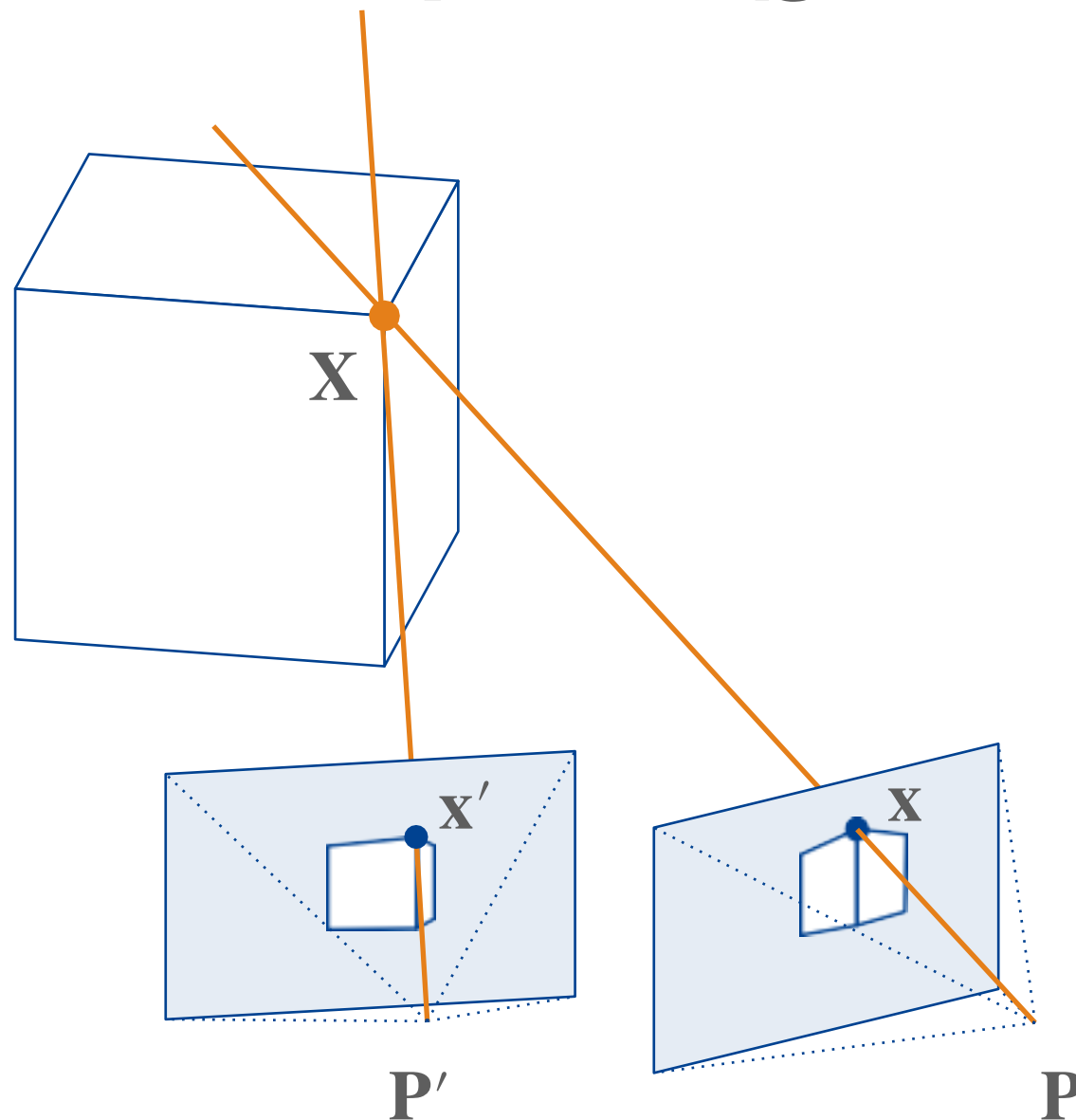


Recap

Triangulation: Computing 3D coordinates of observed points (given their projection matrices)



- Camera with projection matrix P observes 3D point X as

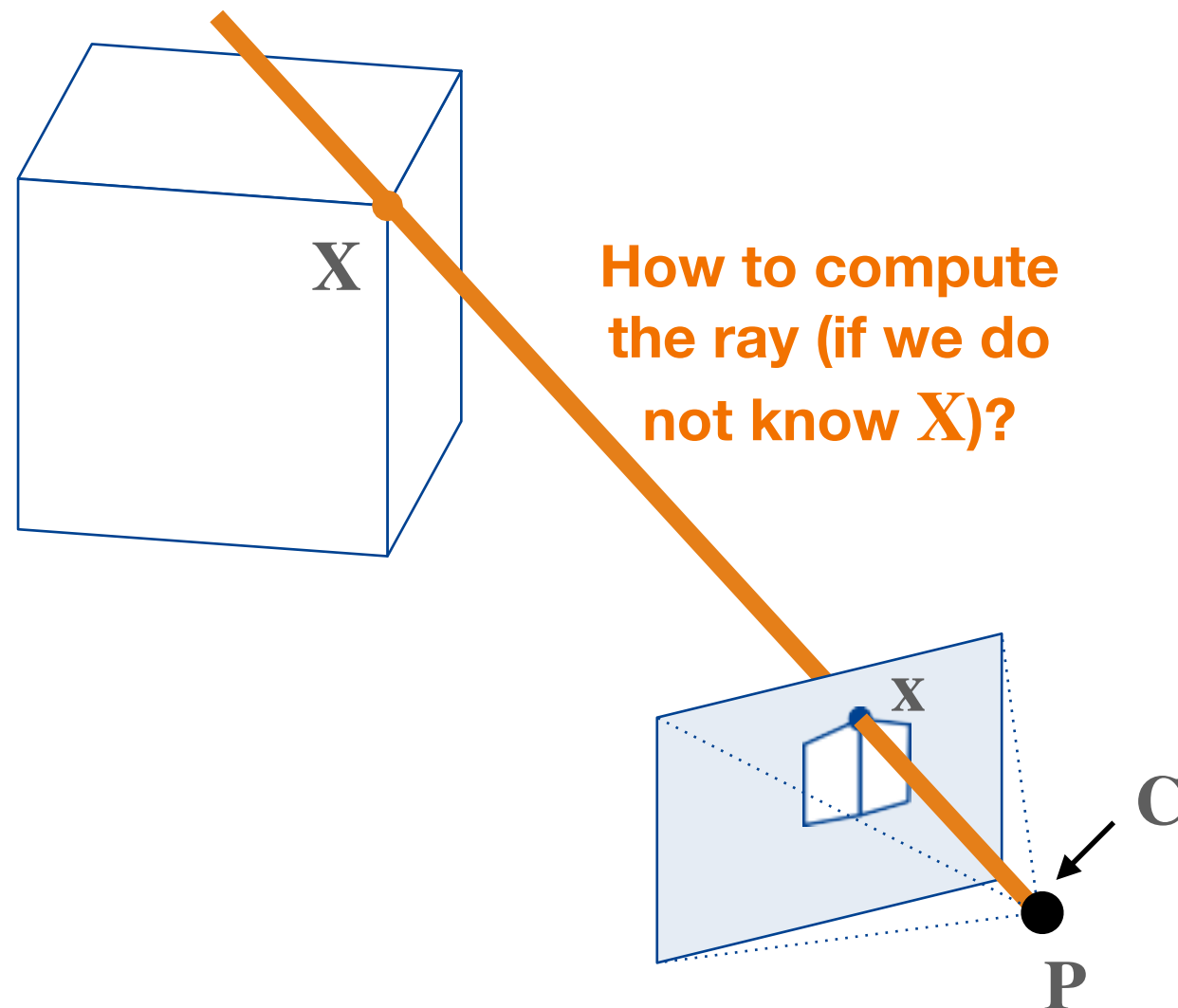
$$x = PX$$

- Second camera with projection matrix P' observes the same point as

$$x' = P'X$$

We need to know the intrinsic and extrinsic camera parameters to do this!

Triangulation constraint: collinearity of the 3D rays



We want to estimate the unknowns X, Y, Z and arrive at a formulation

$$\mathbf{A}\mathbf{X} = 0$$

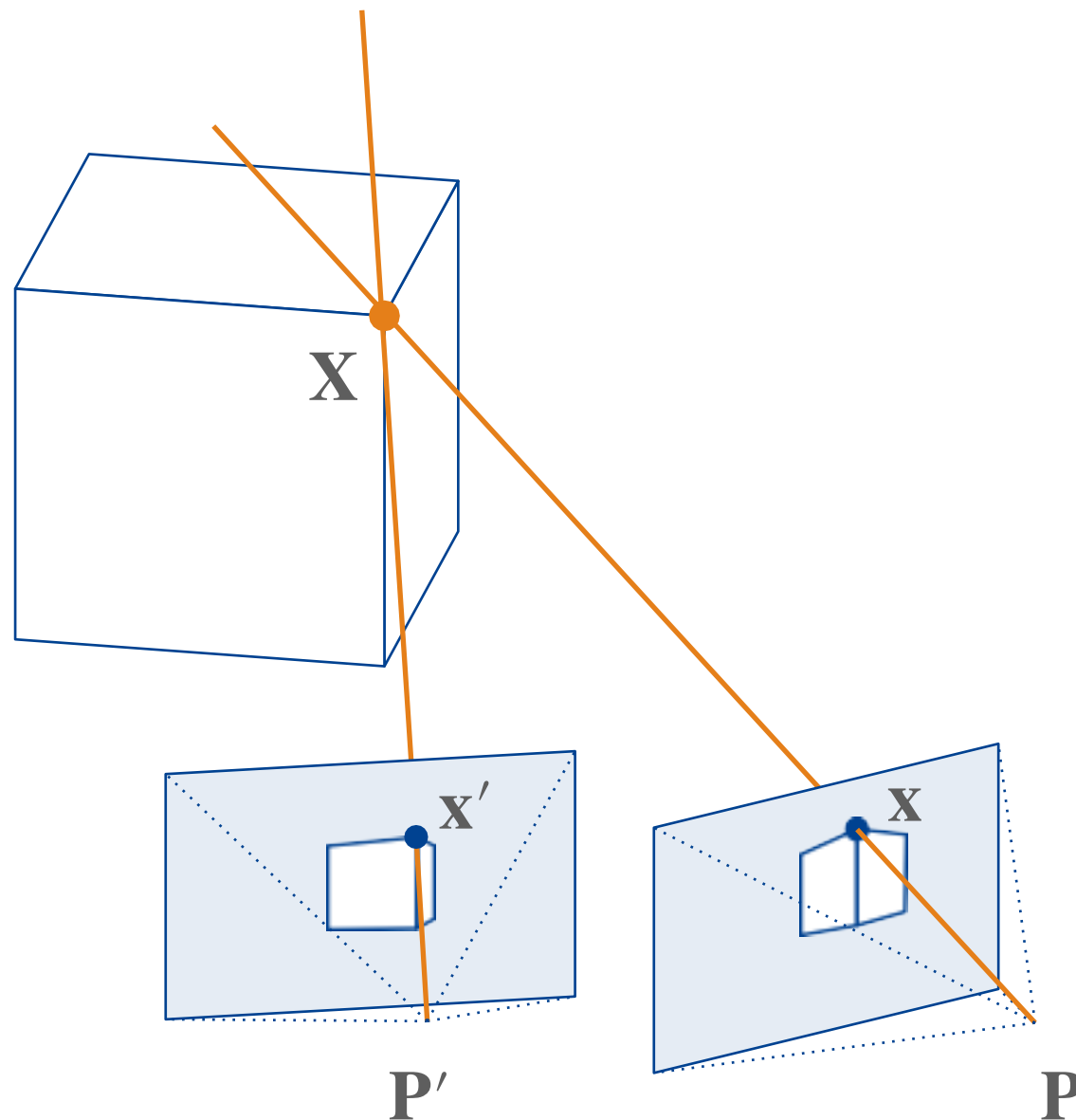
But how to deal with λ ?

Solution: Check for collinearity of $\mathbf{x}$ and $\mathbf{P}\mathbf{X}$ using the cross product

$$\mathbf{x} \times \mathbf{P}\mathbf{X} = 0$$

We get only two constraints from one point observation

Triangulation: Building the linear system



$$\mathbf{A}\mathbf{X} = \mathbf{0}$$

$$\begin{bmatrix} yp_3^\top - p_2^\top \\ p_1^\top - xp_3^\top \\ y'p_3'^\top - p_2'^\top \\ p_1'^\top - x'p_3'^\top \end{bmatrix} \mathbf{X} = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \end{bmatrix}$$

Find a least squares solution using singular value decomposition.

Remember decomposition of projection matrices

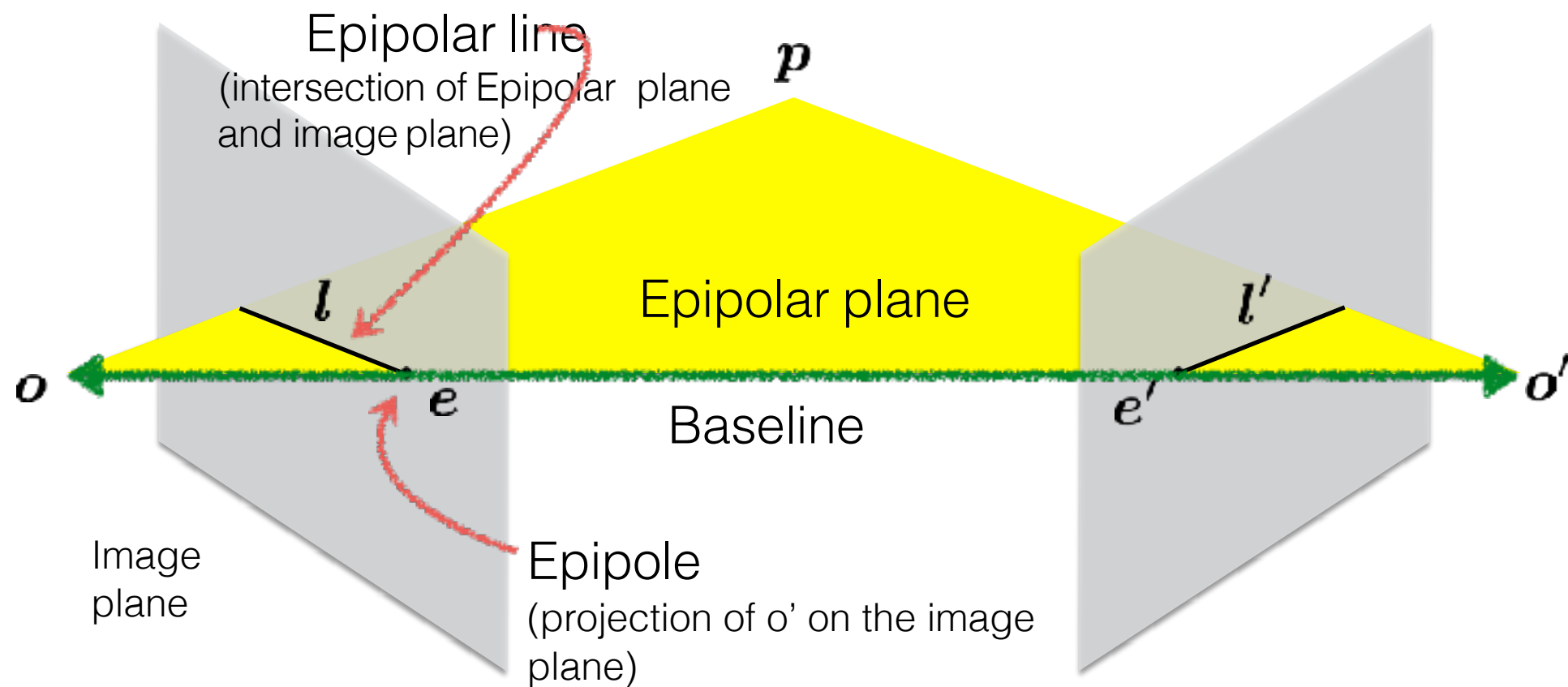
$$\mathbf{\Pi} = \underbrace{\begin{bmatrix} -fs_x & 0 & x'_c \\ 0 & -fs_y & y'_c \\ 0 & 0 & 1 \end{bmatrix}}_{\text{intrinsics}} \underbrace{\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix}}_{\text{projection}} \underbrace{\begin{bmatrix} \mathbf{R}_{3 \times 3} & \mathbf{0}_{3 \times 1} \\ \mathbf{0}_{1 \times 3} & 1 \end{bmatrix} \begin{bmatrix} \mathbf{I}_{3 \times 3} & \mathbf{T}_{3 \times 1} \\ \mathbf{0}_{1 \times 3} & 1 \end{bmatrix}}_{\text{extrinsics}}$$

identity matrix

Note

- Camera calibration gives us intrinsic and extrinsic parameters while observing the calibration pattern.
- When observing other objects with a freely moving camera, we can only use the intrinsics
 - need to estimate the extrinsics for triangulation from the images!
- In the simplest case, we have two images of the same object from the same camera, which has been calibrated.
- Then, the extrinsics can be modeled as the relative rotation and translation between the two image viewpoints, assuming known intrinsic parameters

Epipolar geometry

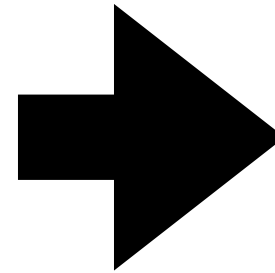


Point sits on epipolar line

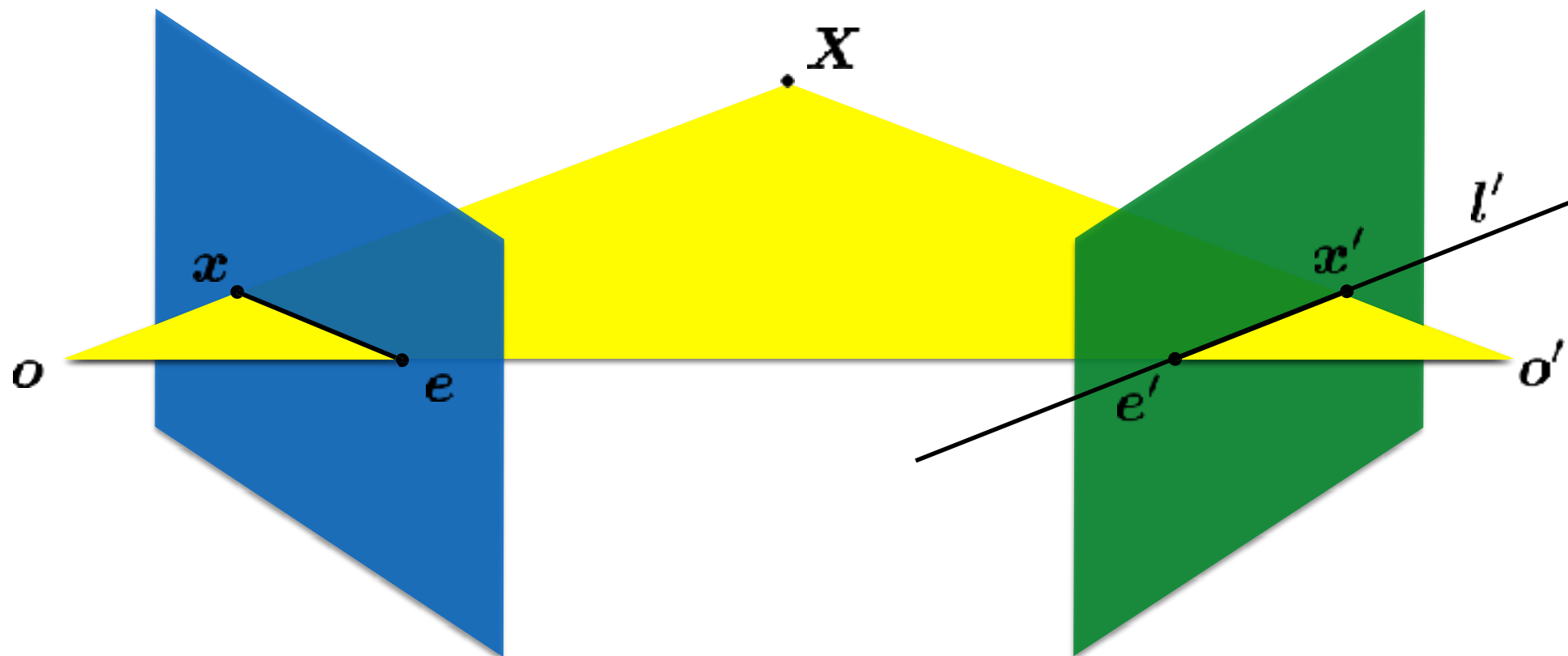
$$\mathbf{x}'^\top \mathbf{l}' = 0$$

$$\mathbf{E}\mathbf{x} = \mathbf{l}'$$

Definition of essential matrix



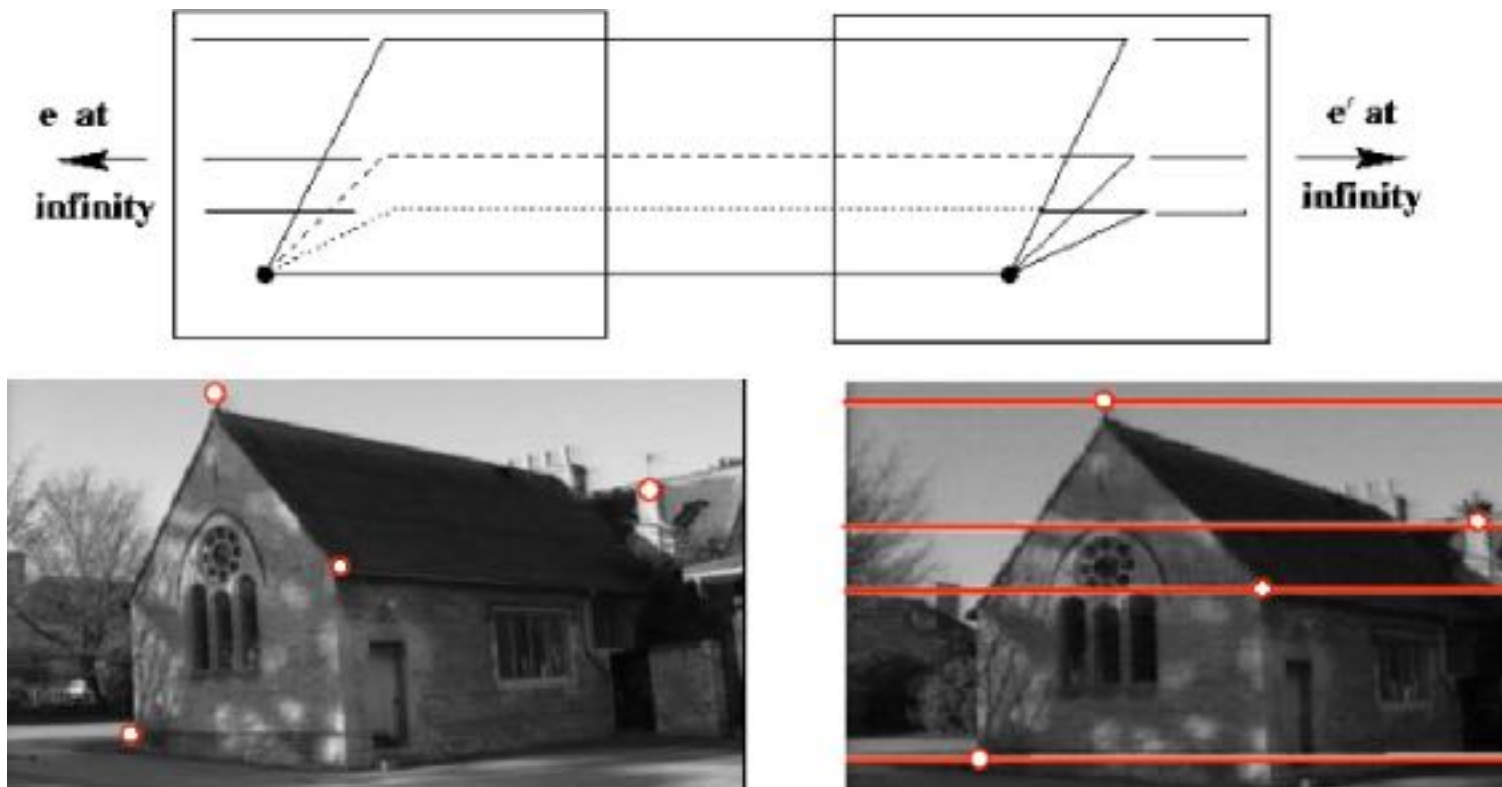
$$\mathbf{x}'^\top \mathbf{E}\mathbf{x} = 0$$



**Stereo disparity
for inferring depth**

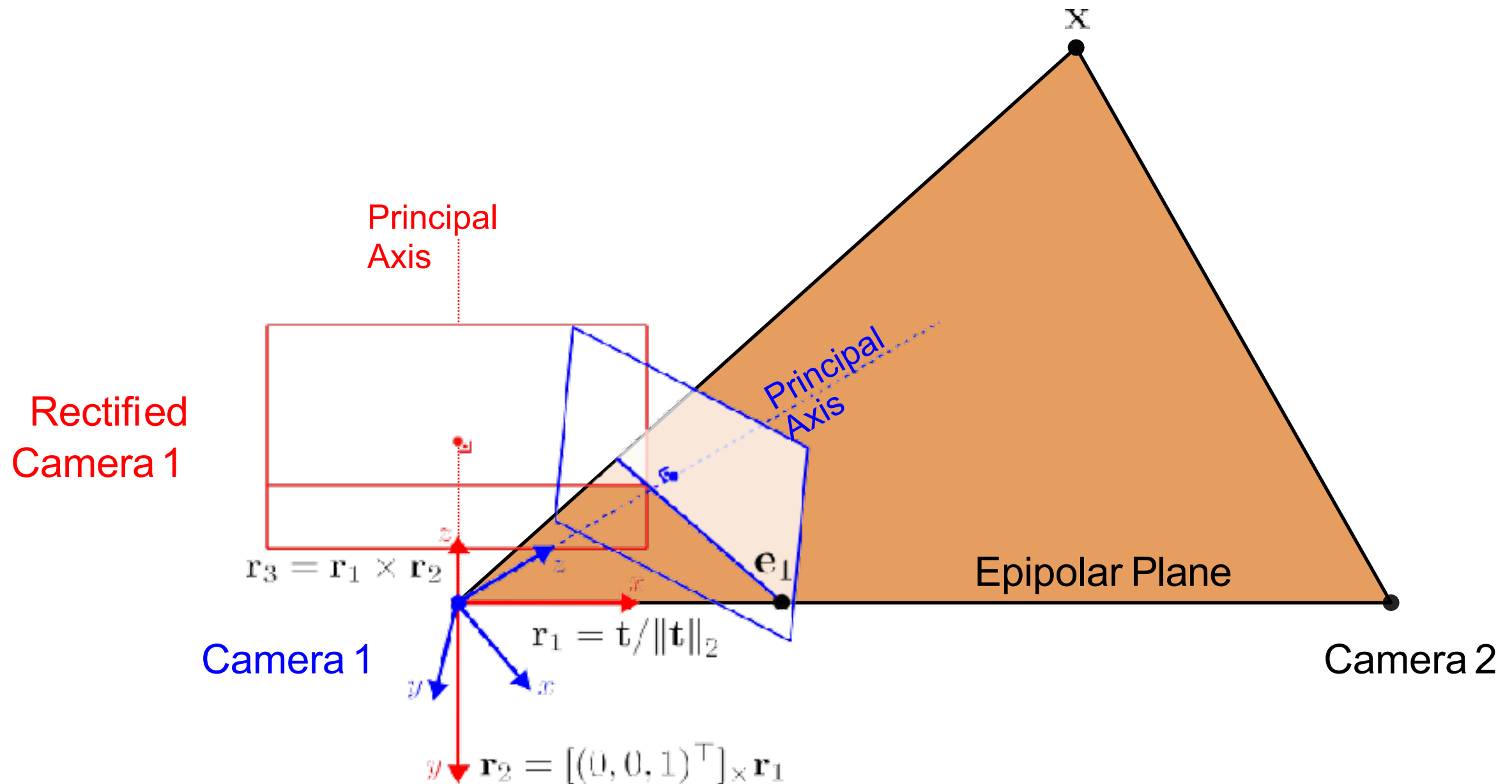
Rectification

What if both cameras face exactly the same direction?



- Image planes are co-planar $\Rightarrow$ Epipoles at infinity, epipolar lines parallel
- Correspondences search along horizontal scanlines (simplifies implementation)

We can enforce a rectified pair by rewarping one camera



Rectification: Example



- Correspondences are located on the **same image row** as the querypoint

Disparity estimation



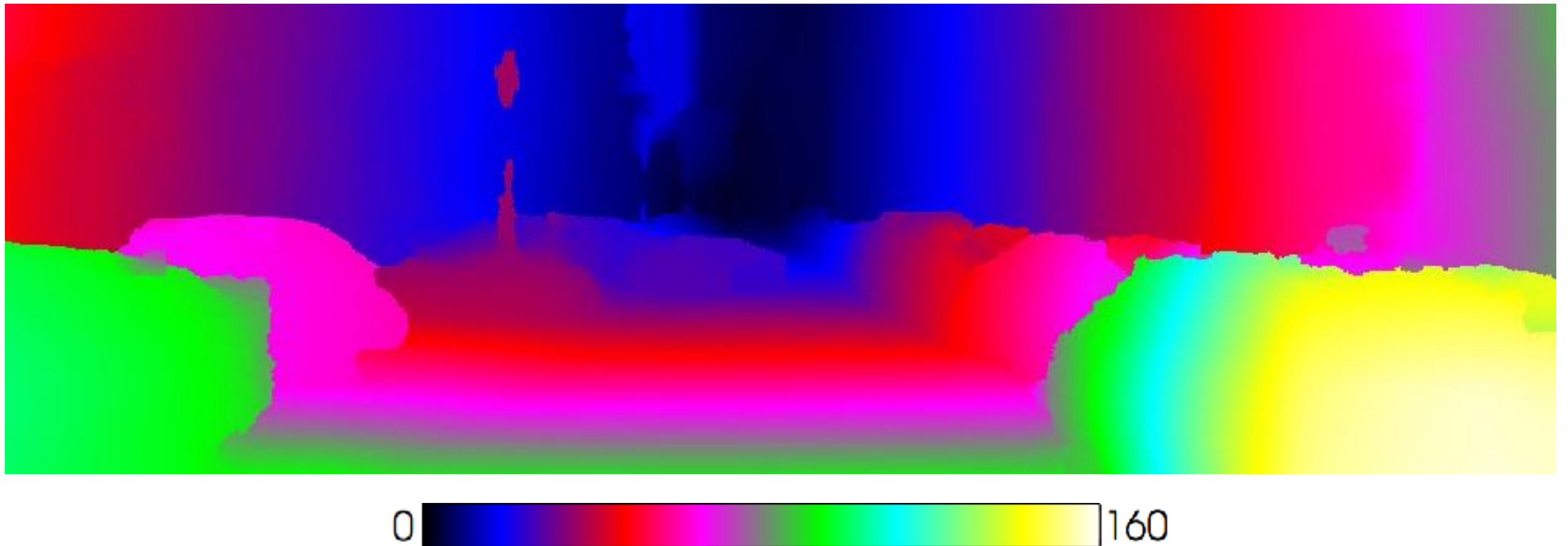
- The relative horizontal displacement is called **disparity** (here: 0-160 px to the left)
- Disparity is anti-proportional to depth (far points move less than closer points)

Disparity estimation



- The relative horizontal displacement is called **disparity** (here: 0-160 px to the left)
- Disparity is anti-proportional to depth (far points move less than closer points)

Disparity estimation



- The relative horizontal displacement is called **disparity** (here: 0-160 px to the left)
- Disparity is anti-proportional to depth (far points move less than closer points)

Disparity to depth

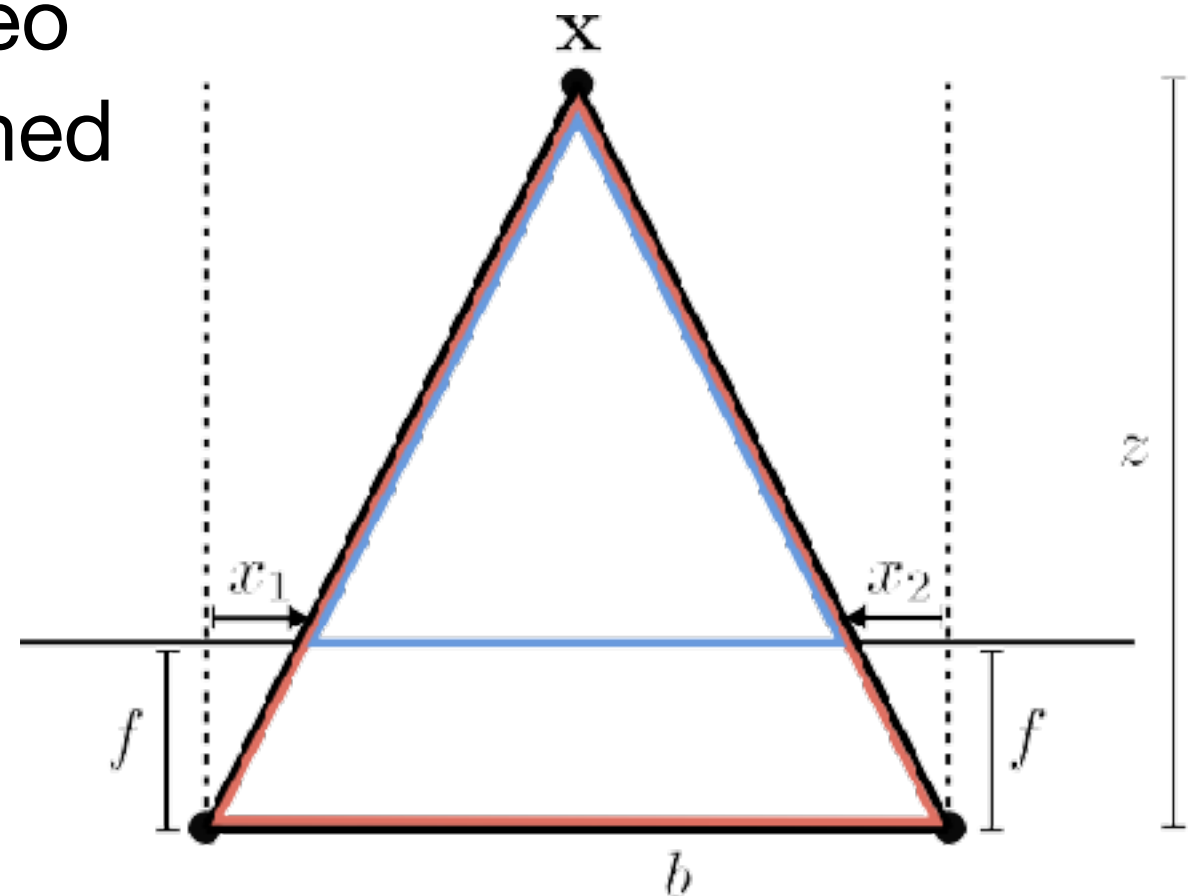
The left and right ray intersect
(shared epipolar plane).

Assume the disparity is $d = x_1 - x_2$.
Given a focal length f and the stereo
baseline b , the depth z is constrained
by

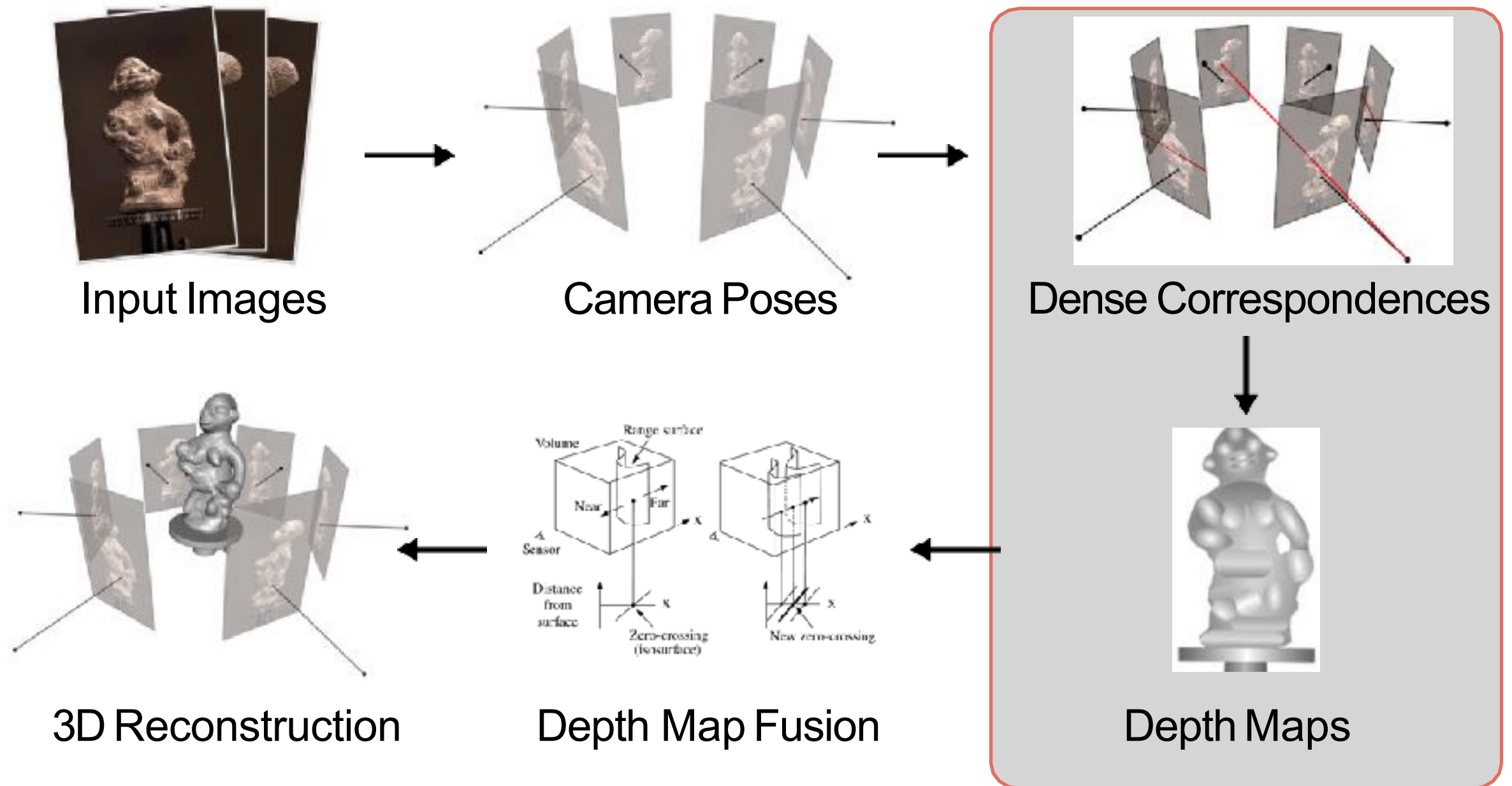
$$\frac{z - f}{b - d} = \frac{z}{b}$$

$$zb - fb = zb - zd$$

$$z = \frac{fb}{d}$$

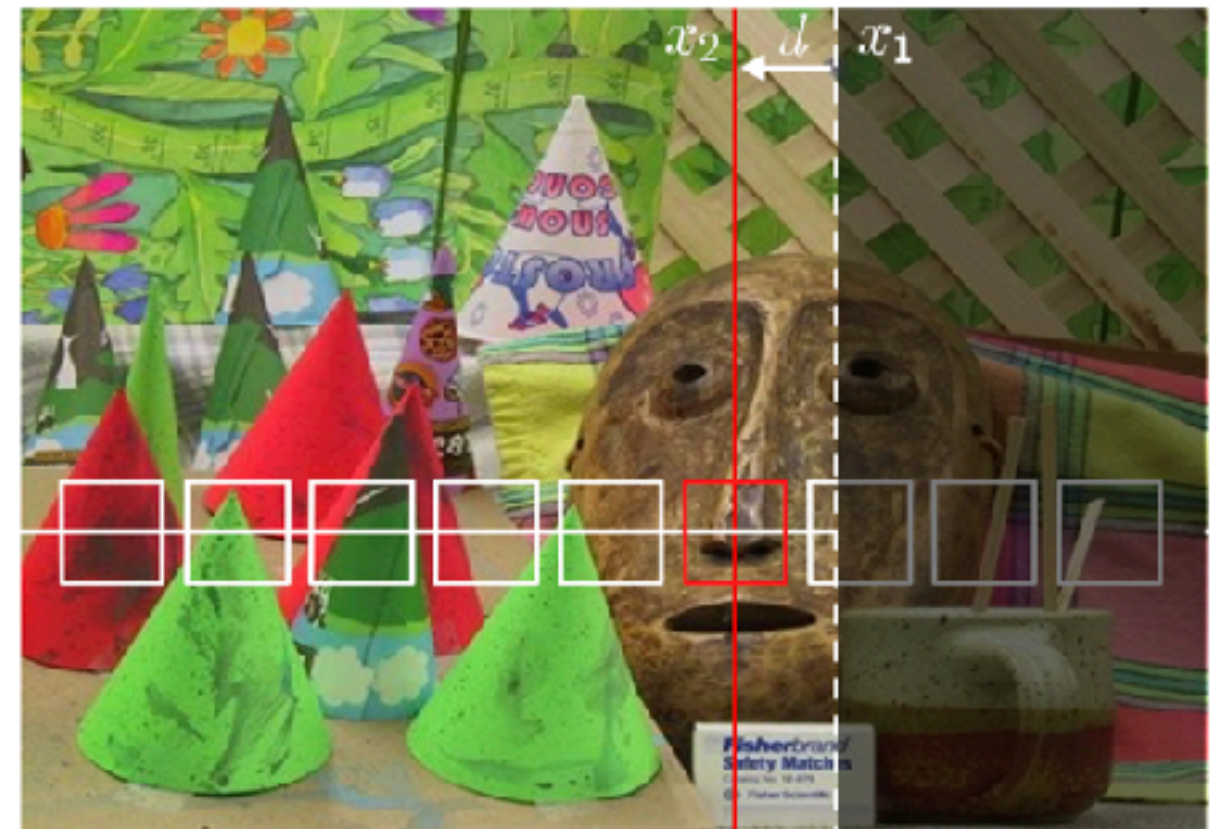


3D Reconstruction Pipeline



Dense stereo matching

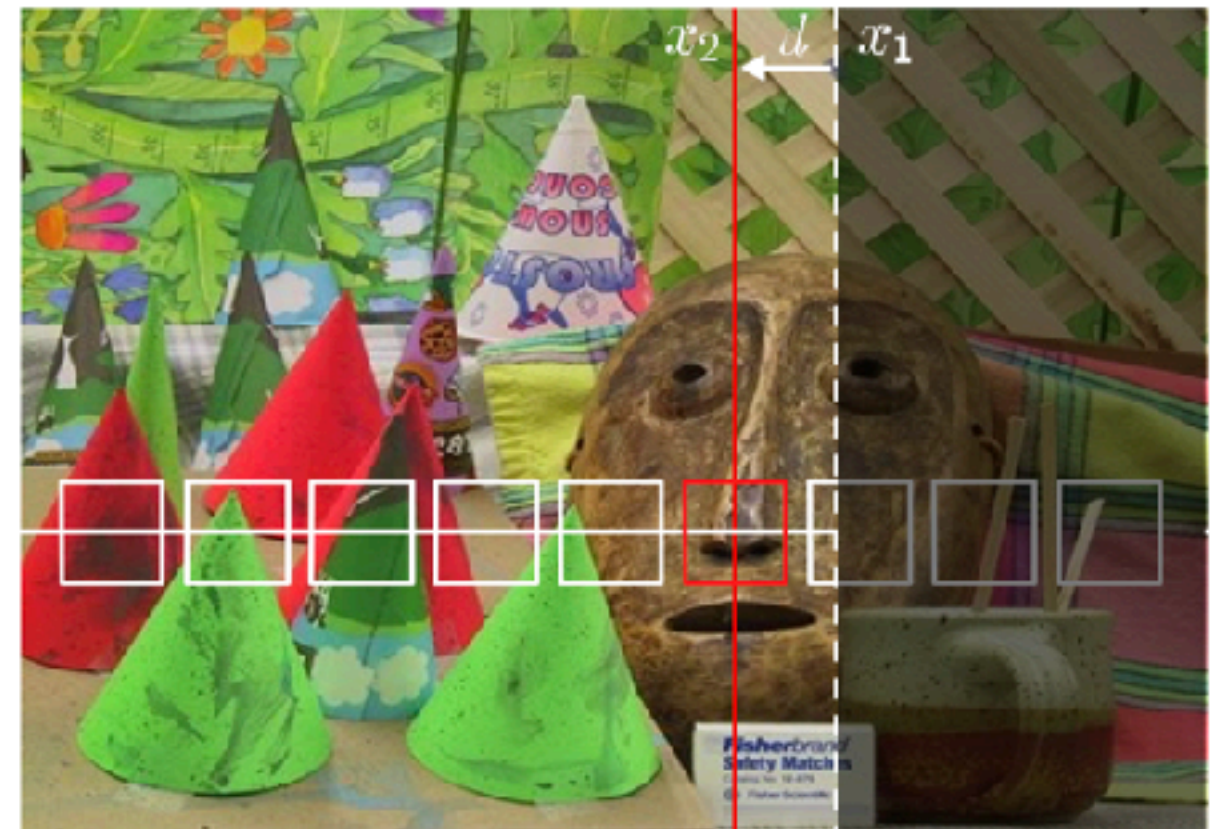
Stereo matching



Given rectified images, we can match a patch in the first image by computing similarities of horizontal shifted patches in the right image.



Stereo matching

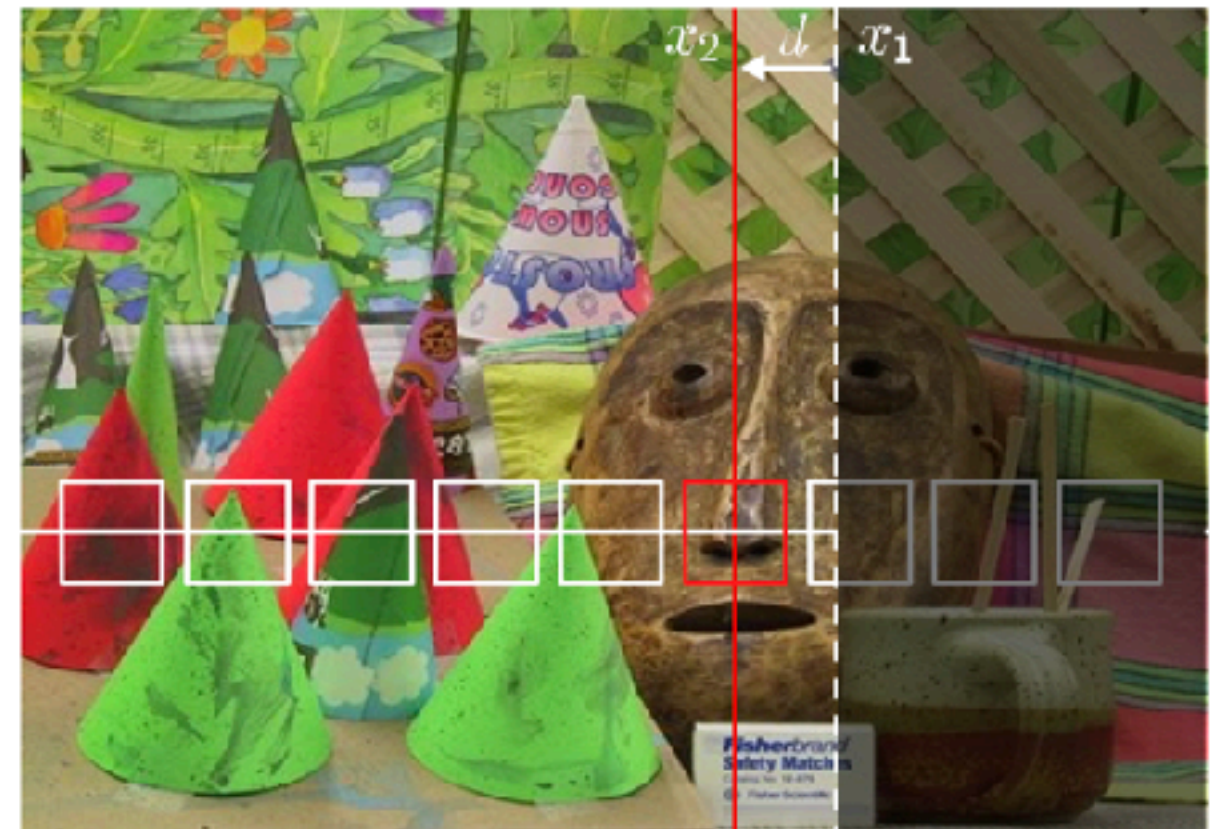


Typical scores:

- Normalized cross correlation
- Means squared error
- Mutual information



Stereo matching

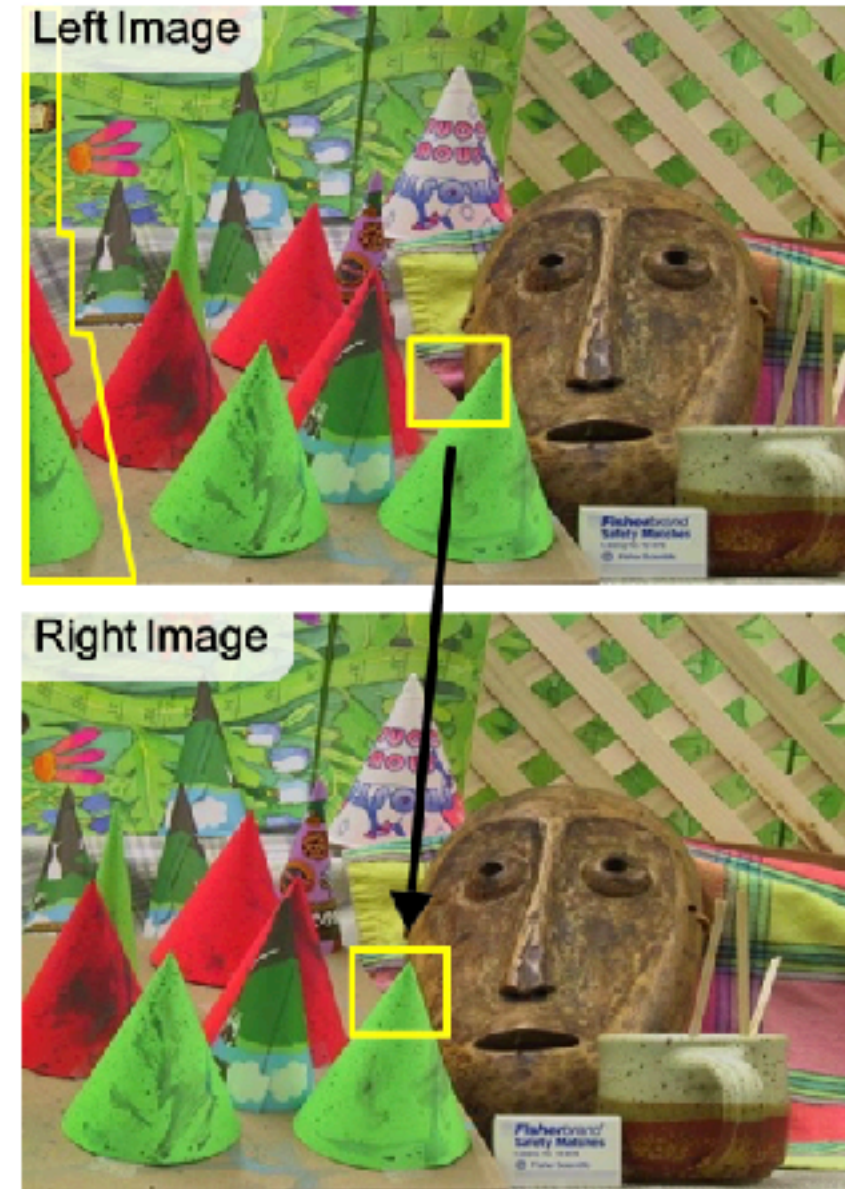
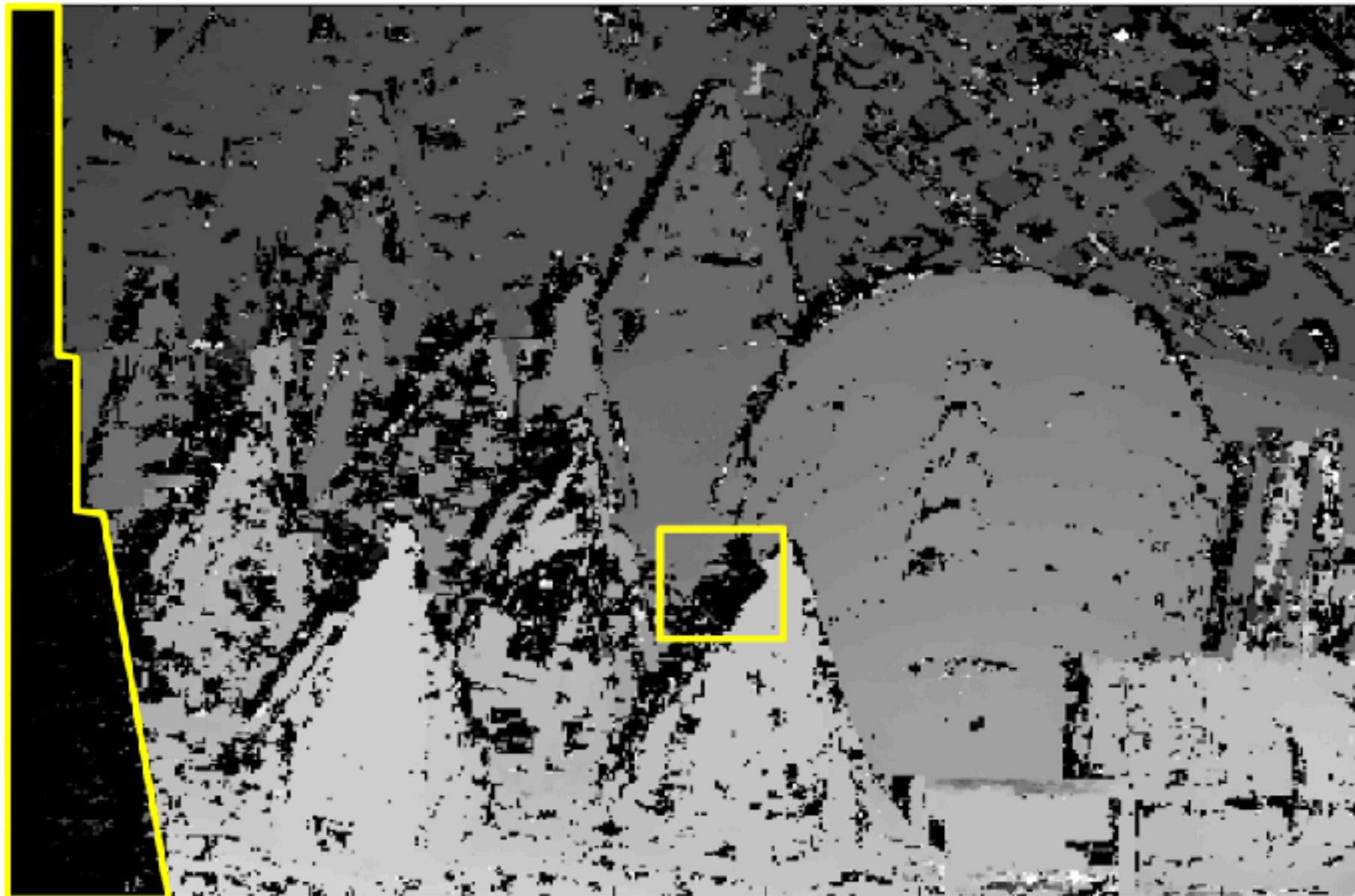


Block matching

- Choose disparity range
- For all pixels, compute best disparity
- Repeat in opposite direction, remove inconsistencies to reduce outliers

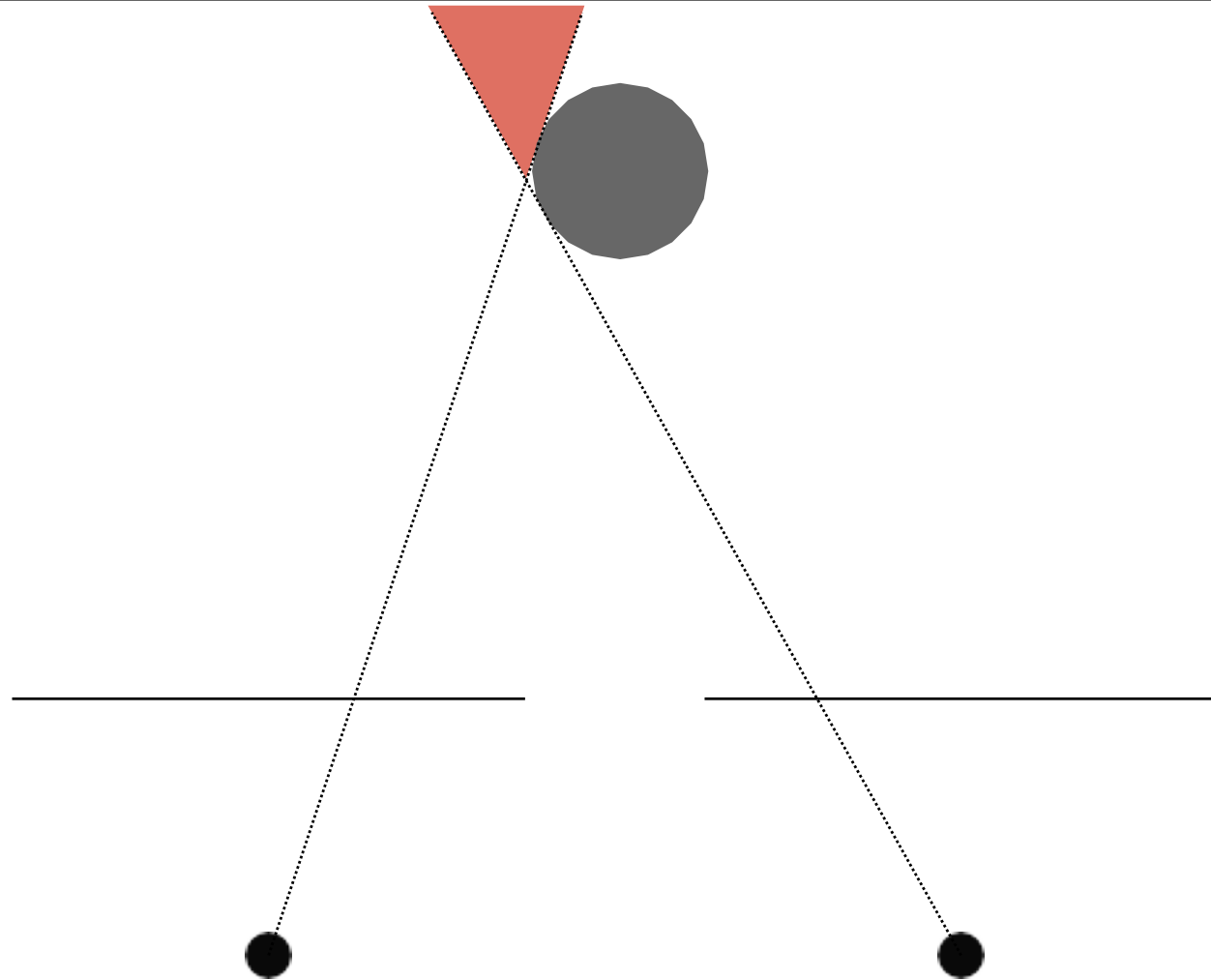


Problem: Half occlusions



Problem: Half occlusions

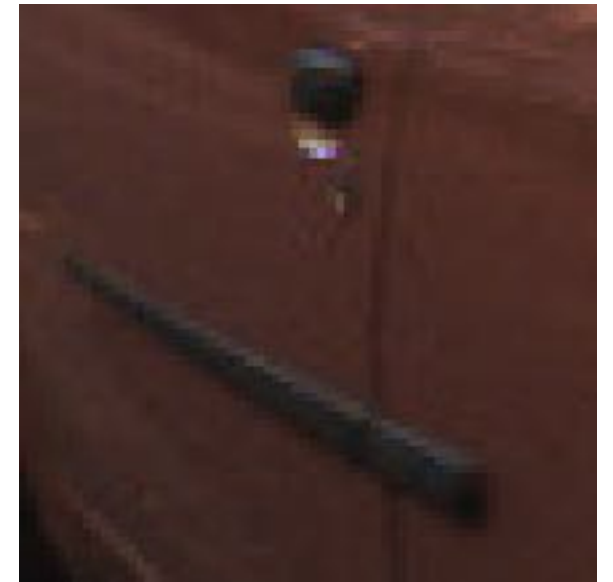
Half Occlusion



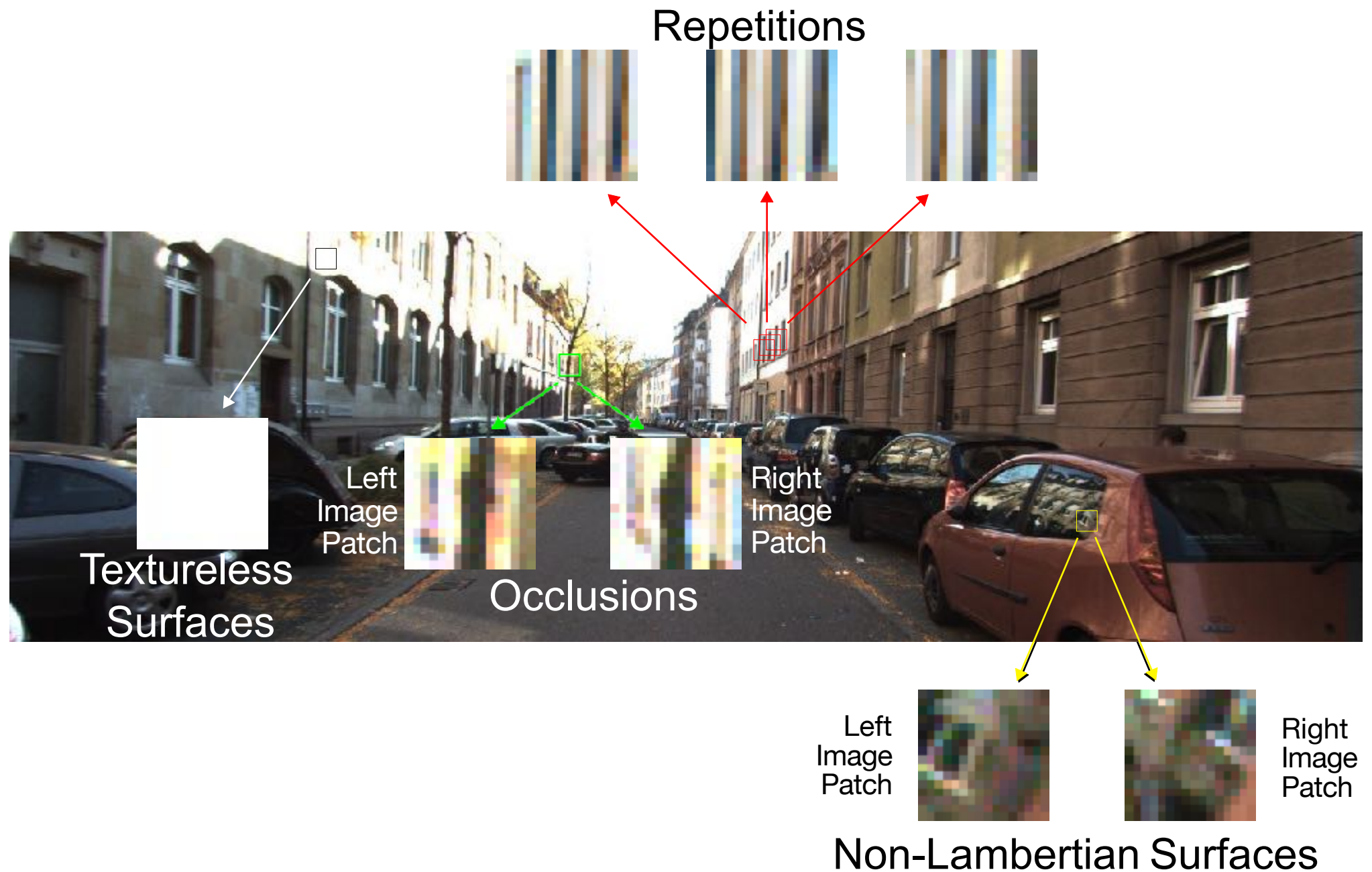
Problem: Slanted surfaces



Problem: Slanted surfaces



Many ambiguities...





Despite all these problems: The „real“ depth is in practice mostly smooth - except at typically rare discontinuities such as object boundaries.

How to incorporate
spatial smoothness into
depth estimation?

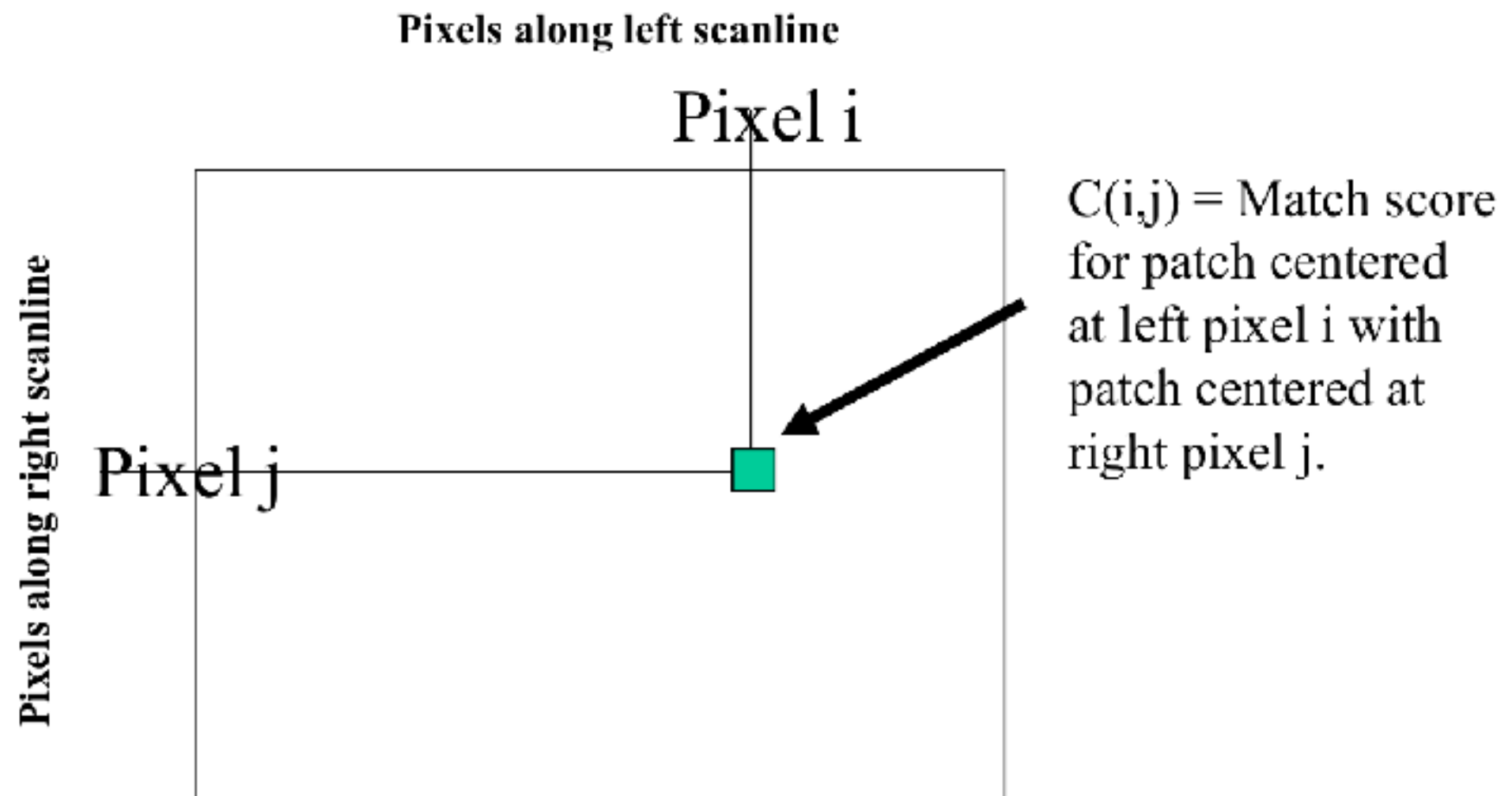
Basic idea

- We want to find the correct disparity at each pixel by matching it to the corresponding point in the other image
 - We match image patches centered at the respective pixels
 - From disparity, we obtain depth
- The match should optimize similarity of the patches
 - this is an independent objective at each pixel
- At the same time, matches of neighboring pixels should usually lead to similar depth estimates
 - this objective refers to results of pairs of neighboring pixels

Dynamic programming for stereo matching

Disparity Space Image

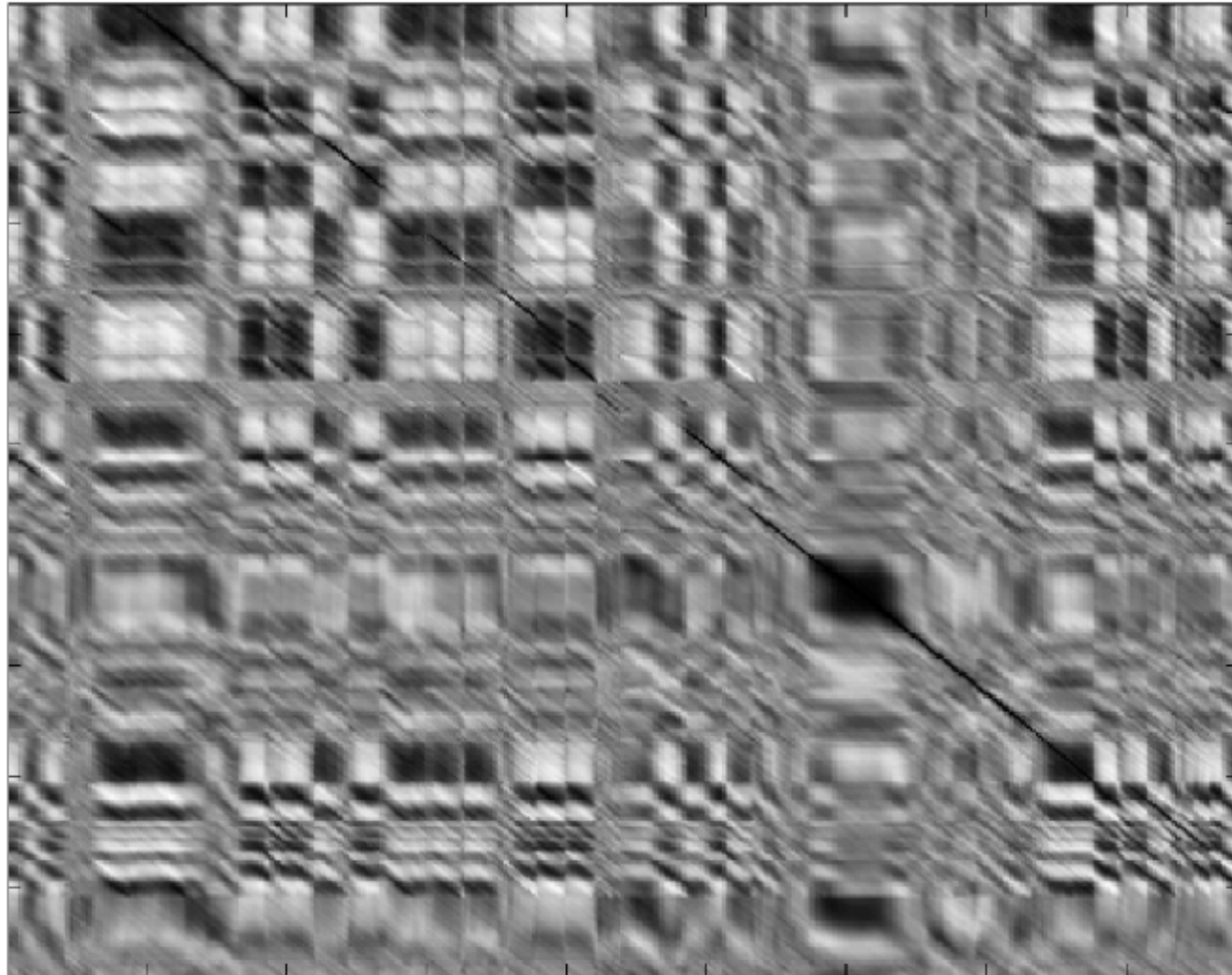
First we introduce the concept of DSI.
The DSI for one row represents pairwise match scores
between patches along that row in the left and right image.



Disparity Space Image

Left scanline

Right scanline



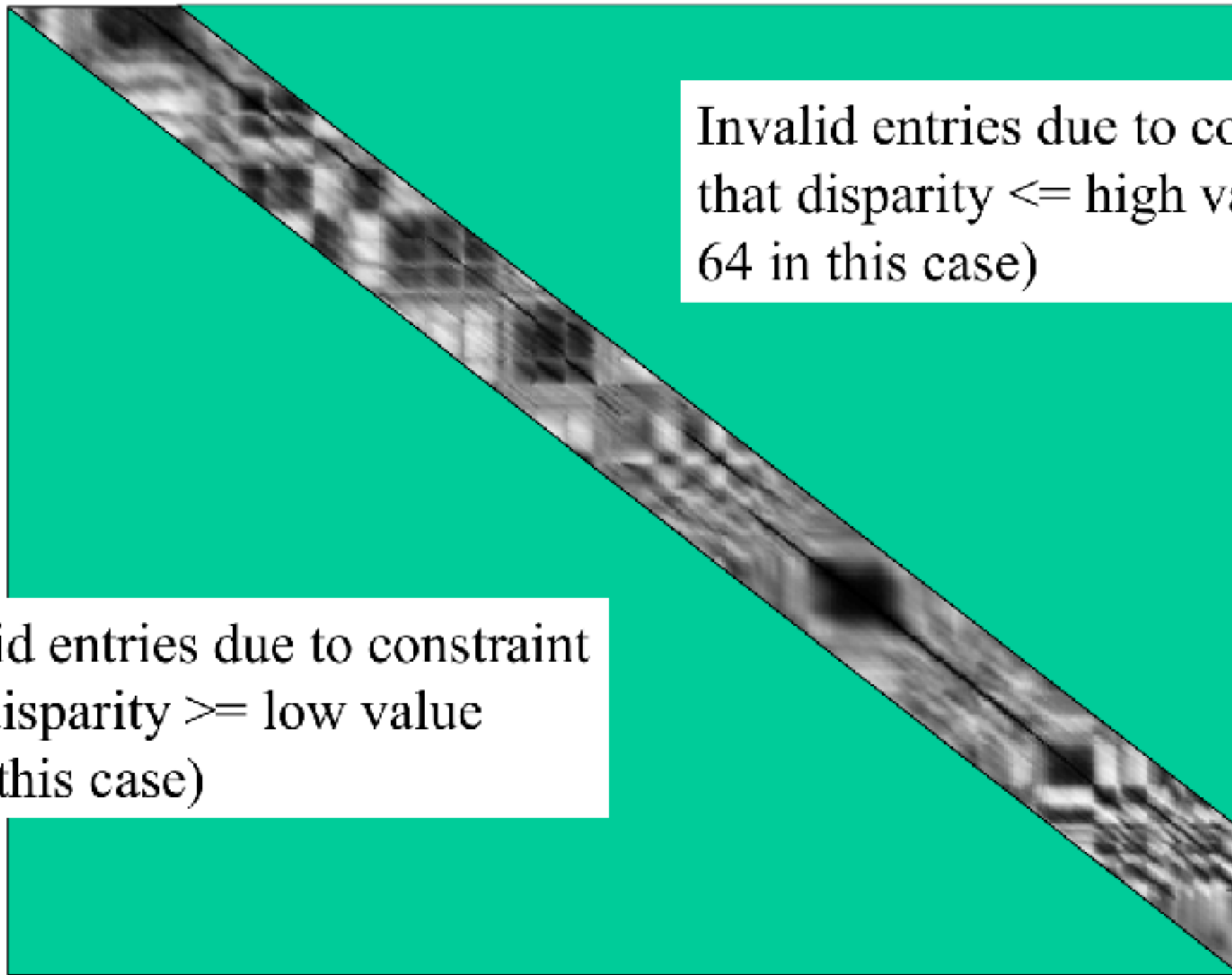
Disparity Space Image

Left scanline

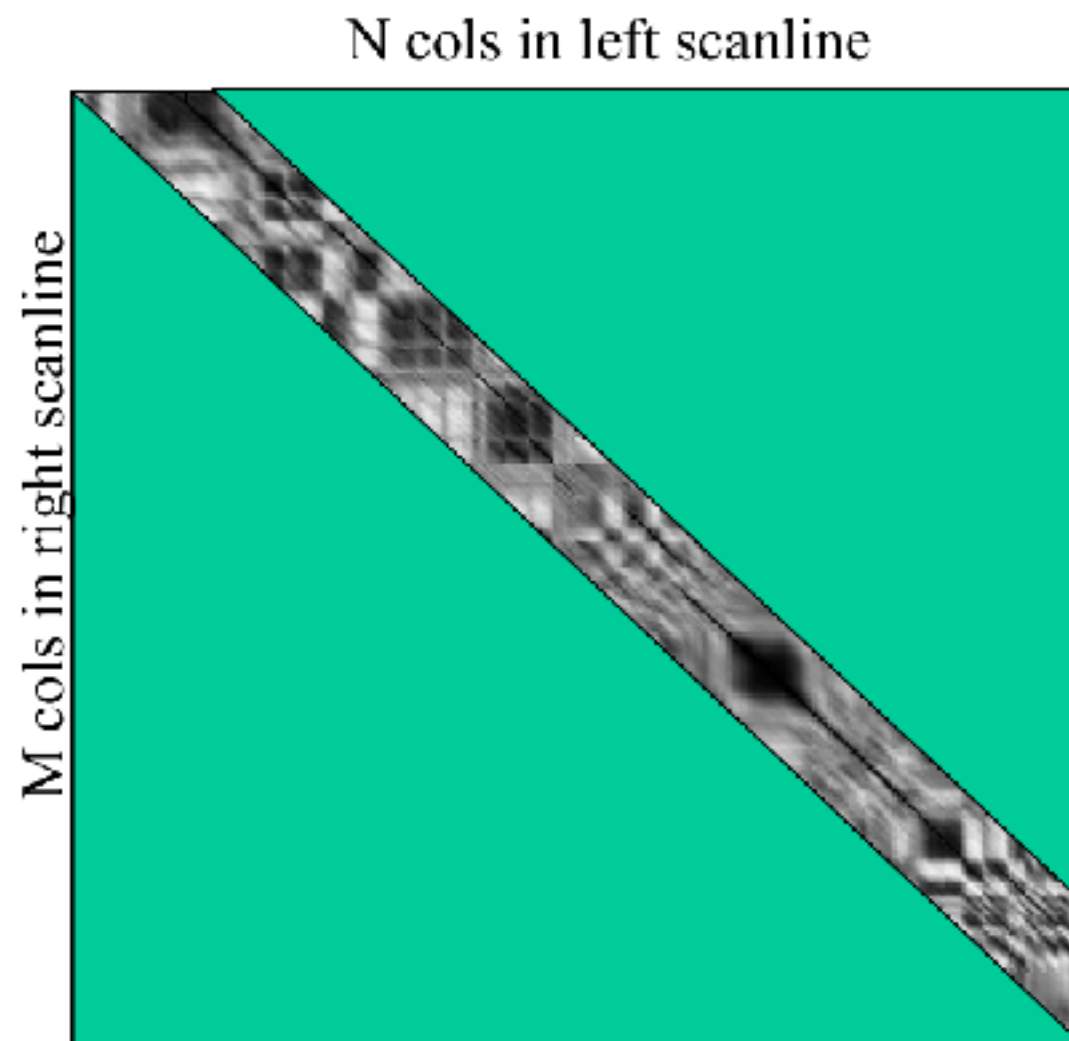
Right scanline

Invalid entries due to constraint
that disparity $\leq$ high value
64 in this case)

Invalid entries due to constraint
that disparity $\geq$ low value
(0 in this case)



Disparity Space Image



If we rearrange the diagonal band of valid values into a rectangular array (in this case of size $64 \times N$), that is what is traditionally known as the DSI

However, I'm going to keep the full image around, including invalid values (I think it is easier to understand the pixel coordinates involved)

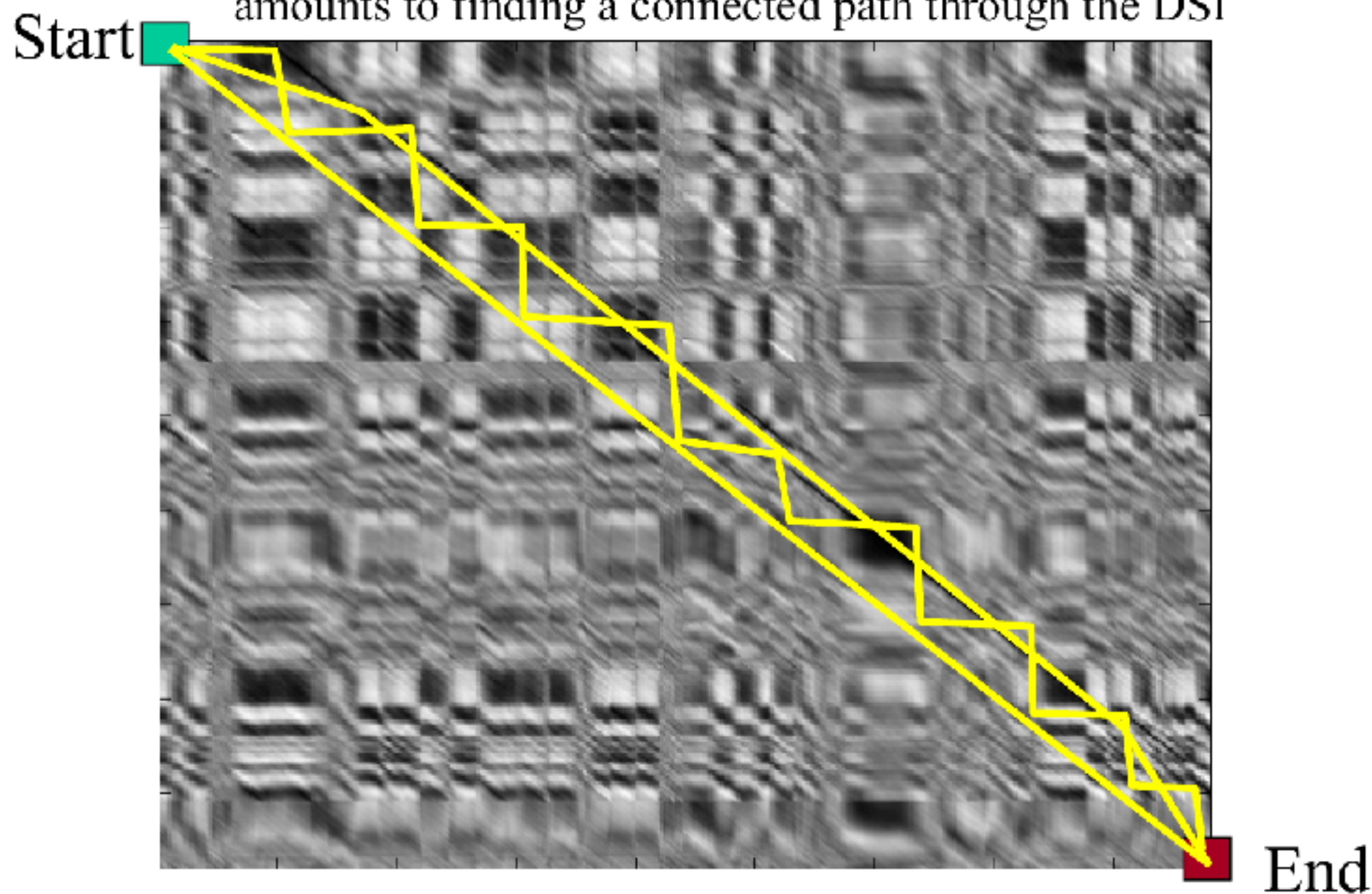
Disparity
(e.g. 64)

coordinate in left scanline (e.g. N)

Disparity Space Image

DSI and Scanline Consistency

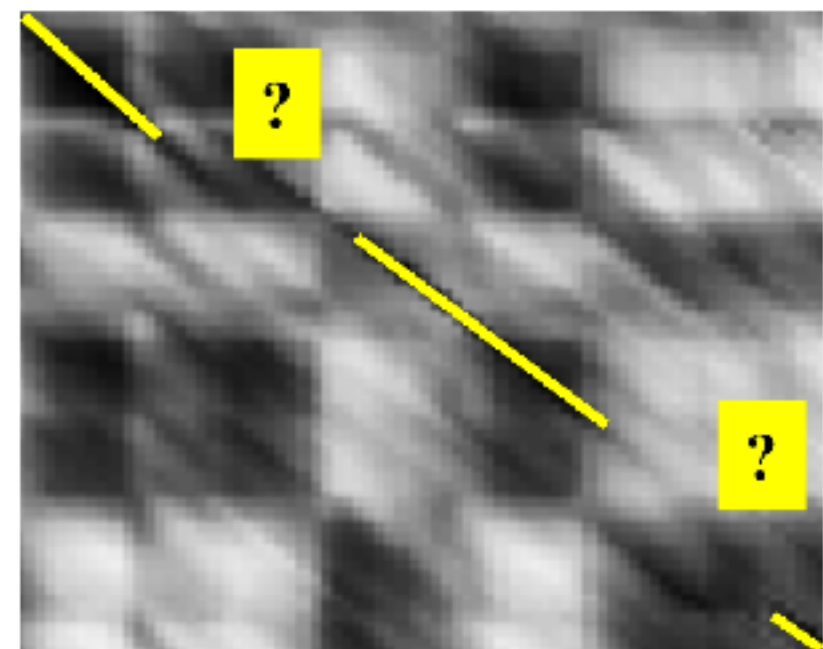
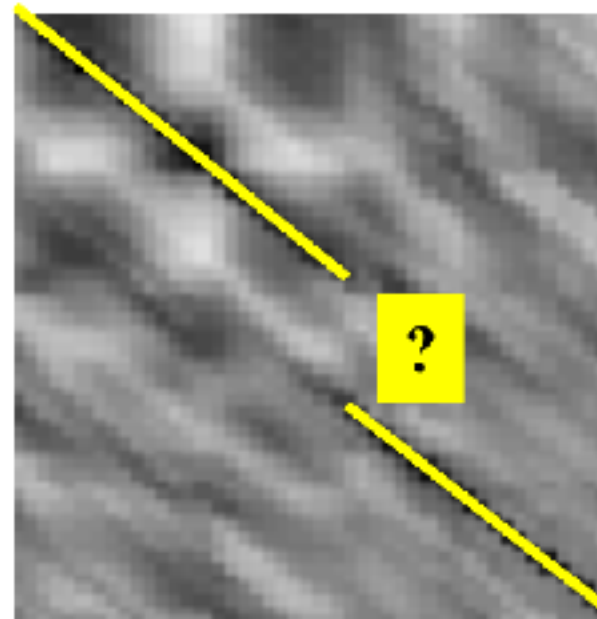
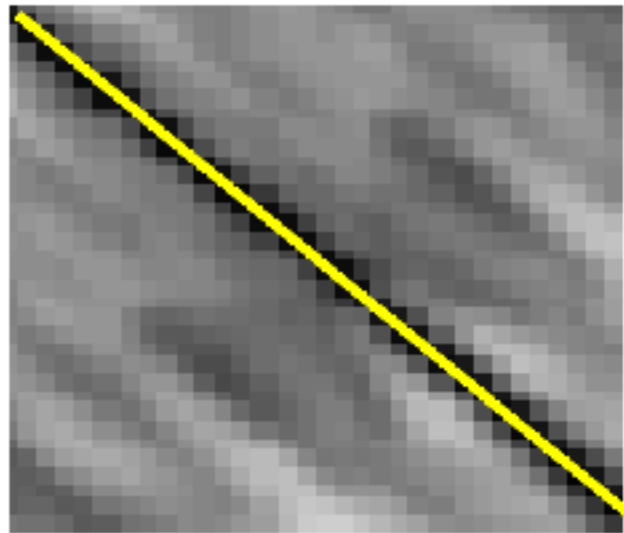
Assigning disparities to all pixels in left scanline now amounts to finding a connected path through the DSI



Lowest Cost Path

We would like to choose the “best” path.

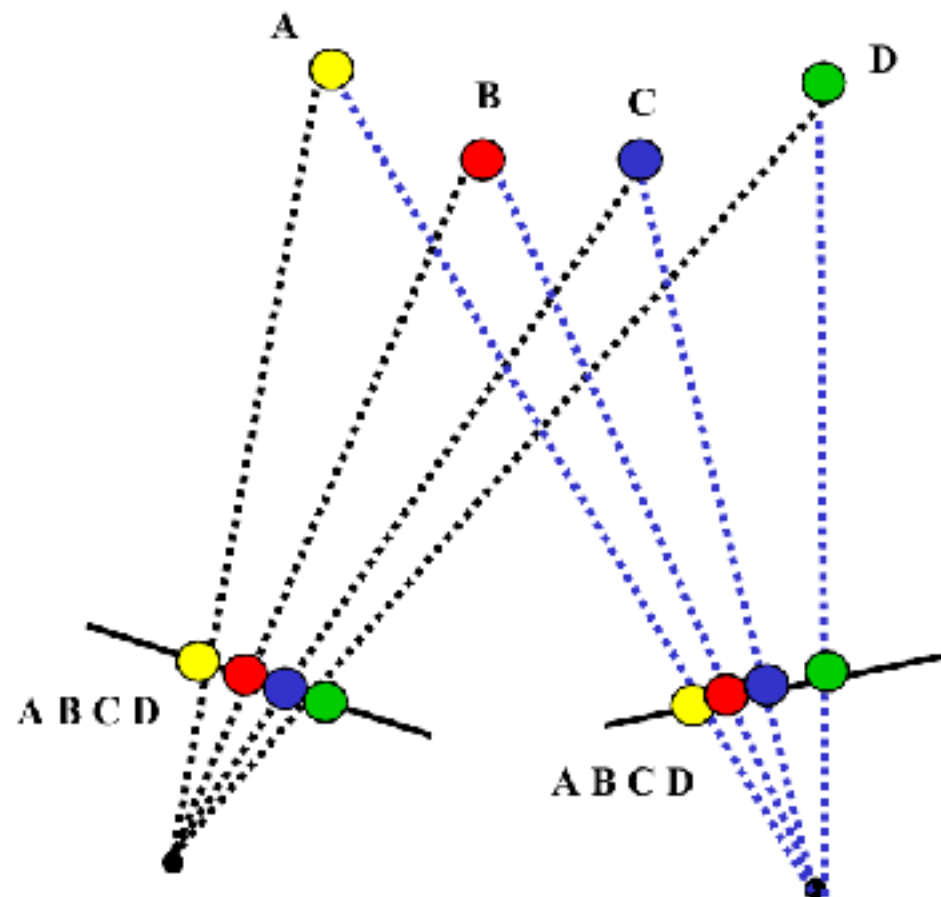
Want one with lowest “cost” (Lowest sum of dissimilarity scores along the path)



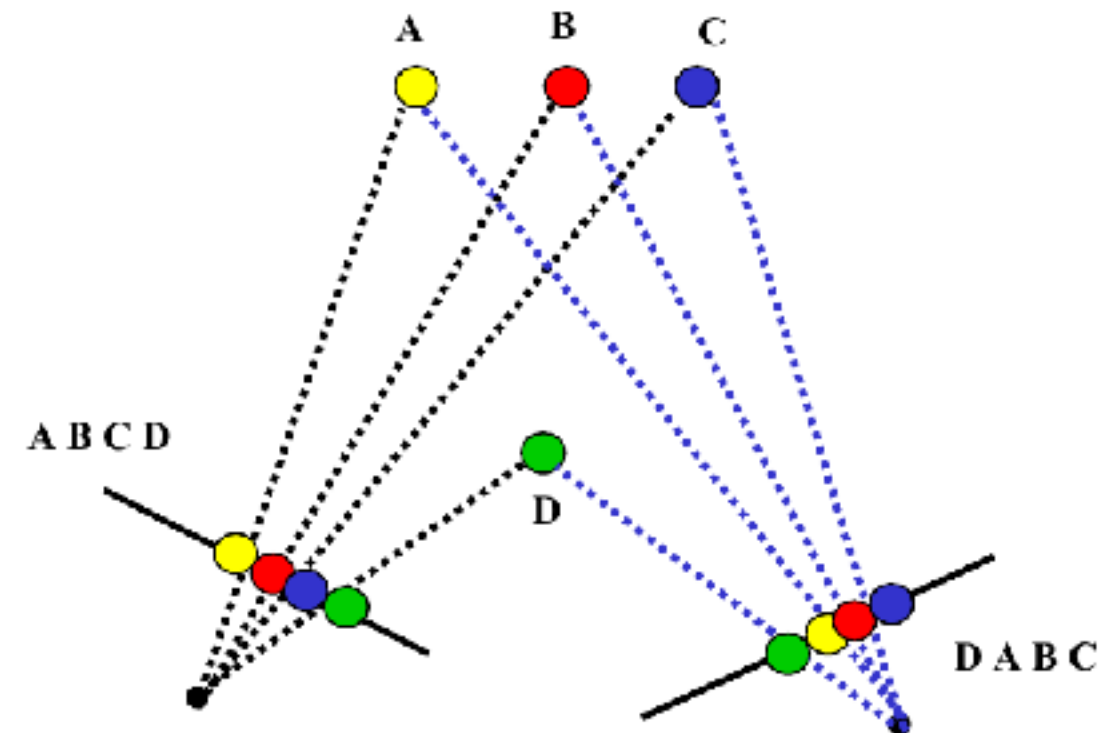
Constraints on Path

It is common to impose an ordering constraint on the path. Intuitively, the path is not allowed to “double back” on itself.

Ordering Constraint

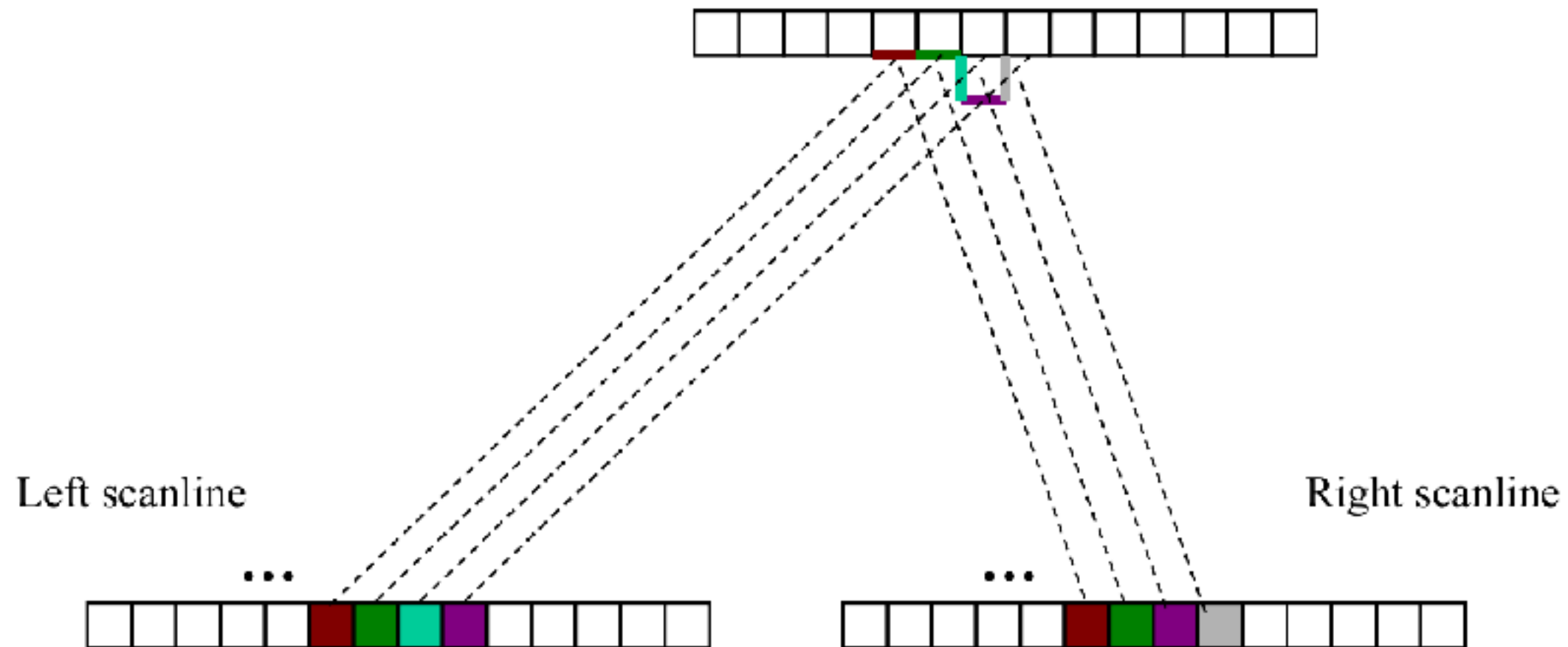


Ordering constraint...

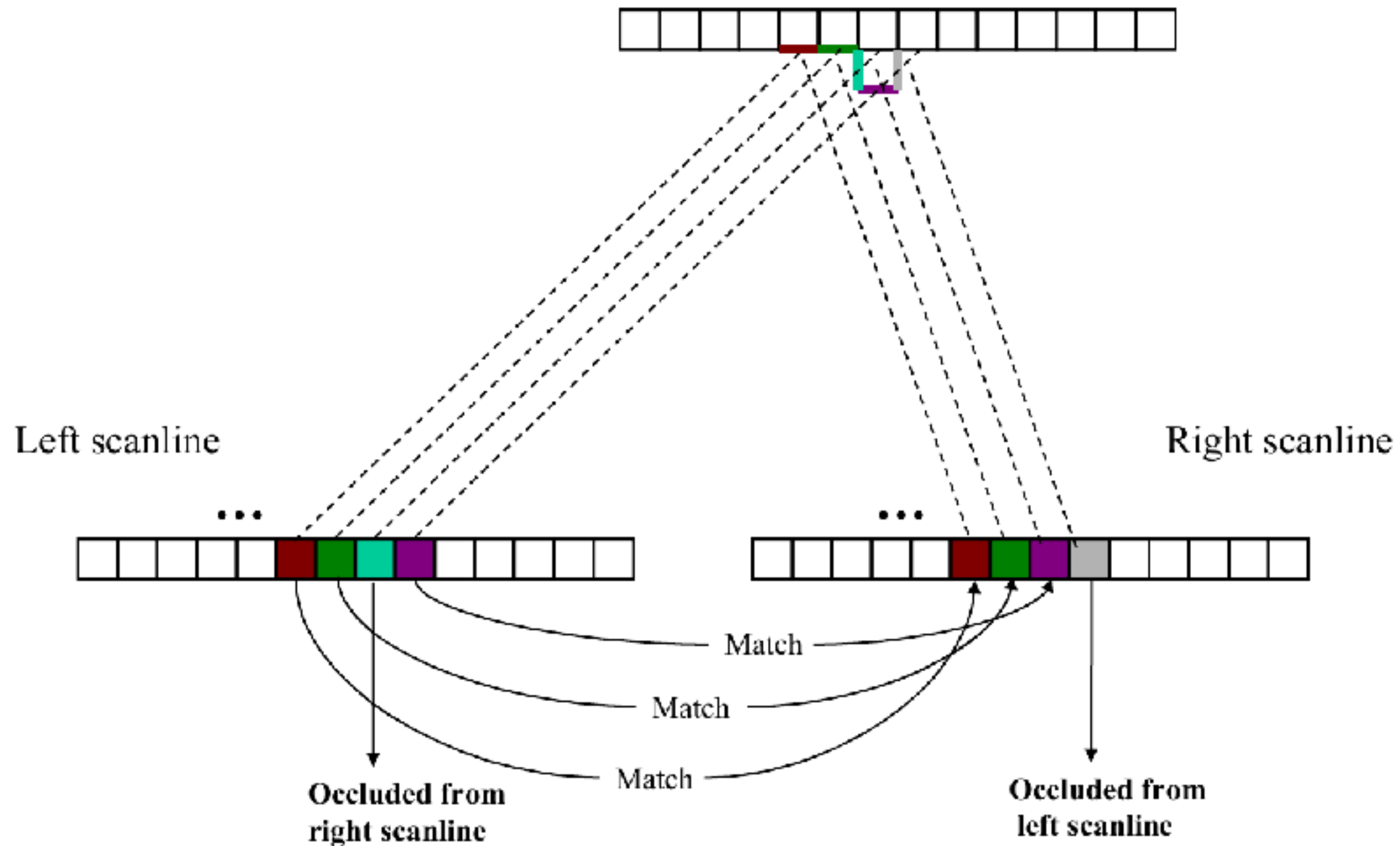


...and its failure

Dealing with Occlusions



Dealing with Occlusions

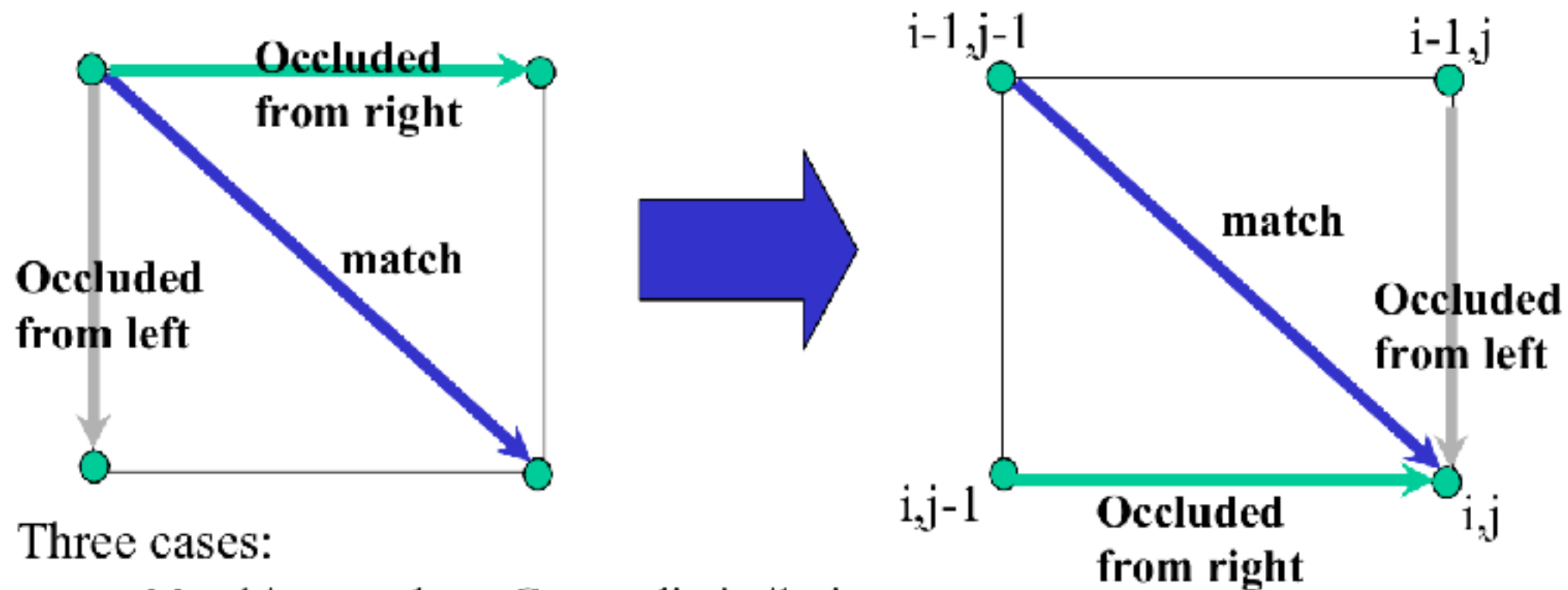


An Optimal Scanline Strategy

- We want to find best path, taking into account ordering constraint and the possibility of occlusions.

**Algorithm we will discuss now is from
Cox, Hingorani, Rao, Maggs, “A Maximum
Likelihood Stereo Algorithm,” Computer
Vision and Image Understanding, Vol 63(3),
May 1996, pp.542-567.**

Cox et.al. Stereo Matching



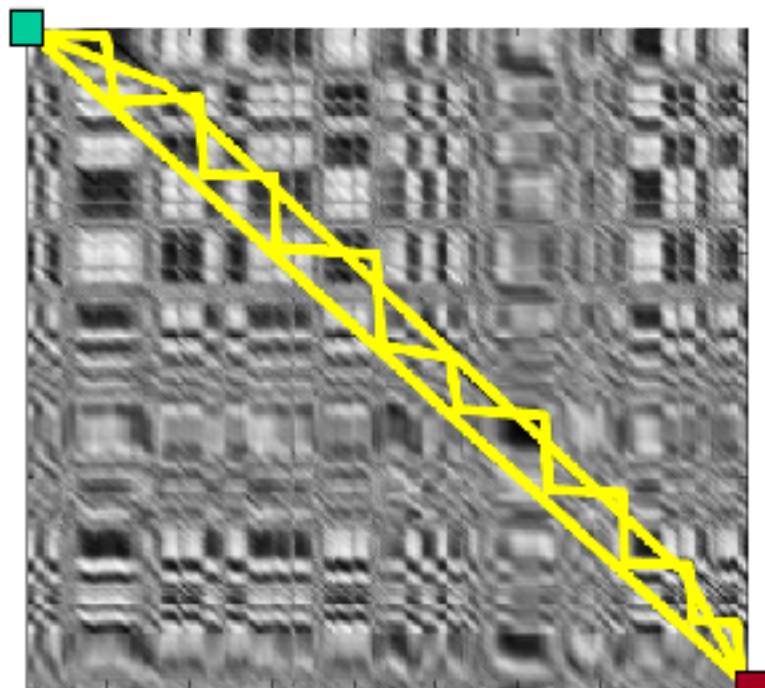
Three cases:

- Matching patches. Cost = dissimilarity score
- Occluded from right. Cost is some constant value.
- Occluded from left. Cost is some constant value.

$$C(i,j) = \min([C(i-1,j-1) + \text{dissimilarity}(i,j) \\ C(i-1,j) + \text{occlusionConstant}, \\ C(i,j-1) + \text{occlusionConstant}]);$$

Cox et.al. Stereo Matching

Start



End

Recap: want to find lowest cost path from upper left to lower right of DSI image.

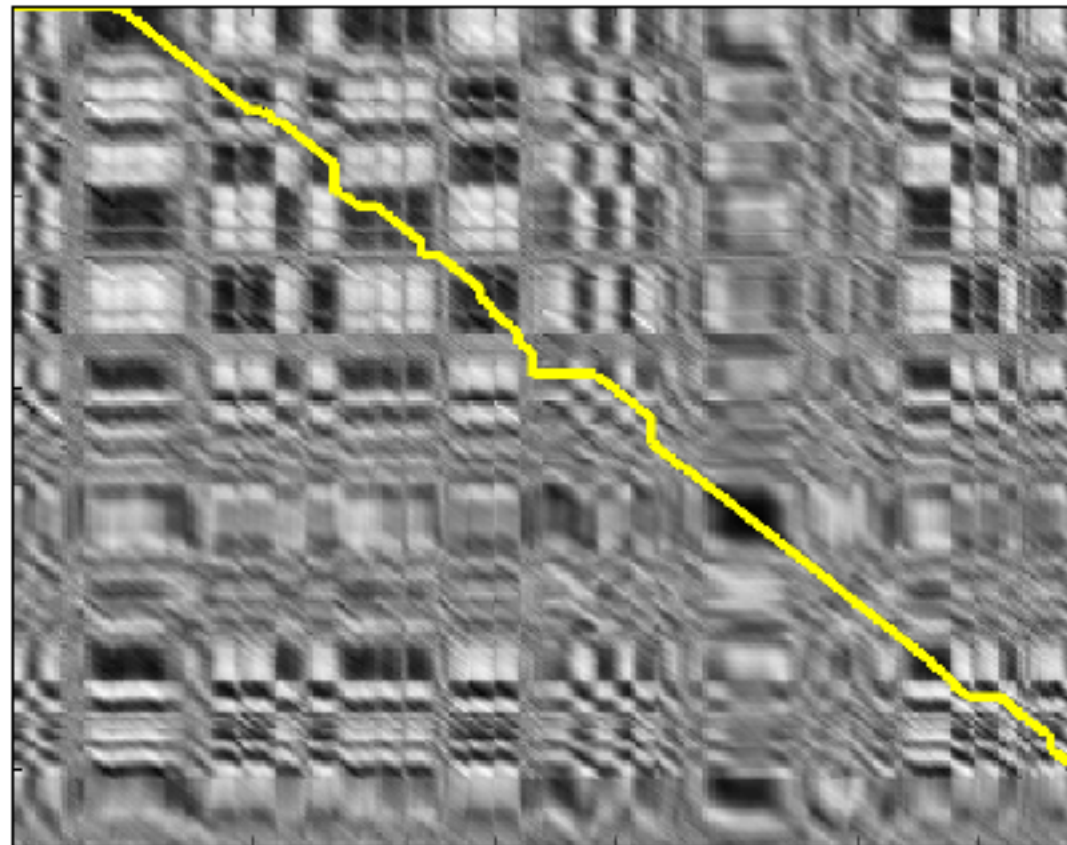
At each point on the path we have three choices: step left, step down, step diagonally.

Each choice has a well-defined cost associated with it.

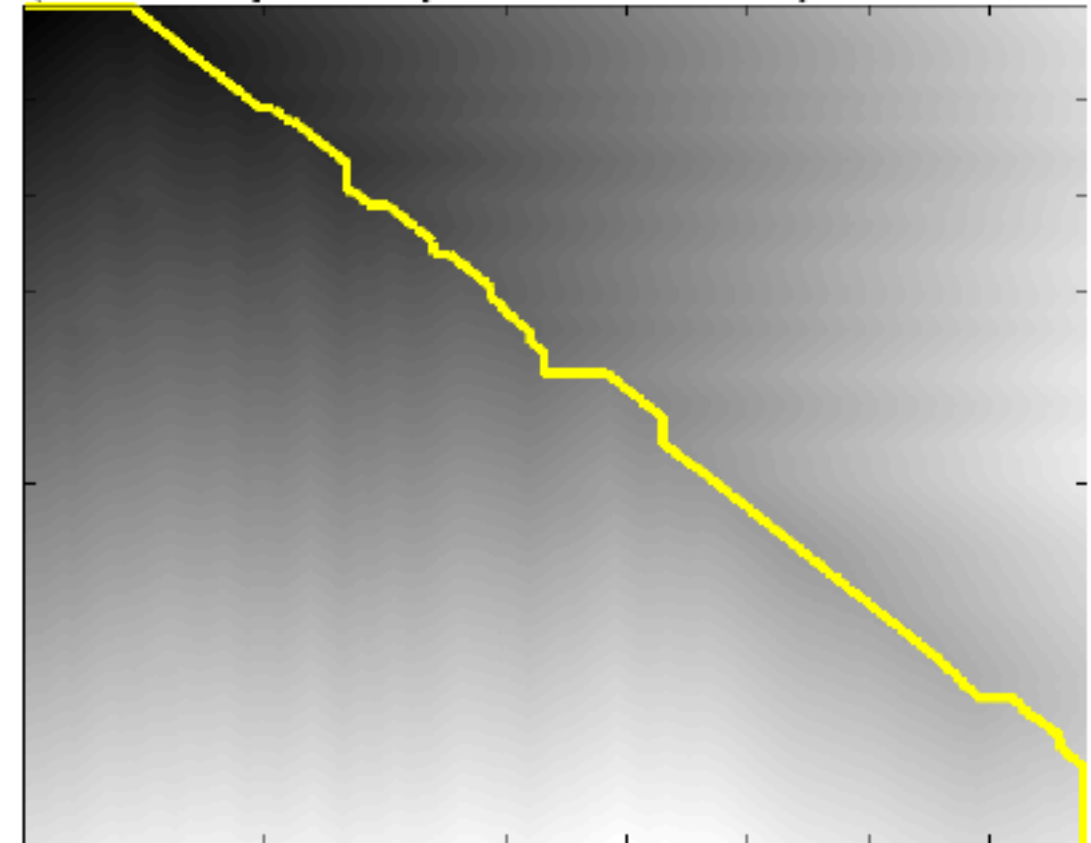
This problem just screams out for Dynamic Programming!
(which, indeed, is how Cox et.al. solve the problem)

Real Scanline Example

DSI



DP cost matrix
(cost of optimal path from each point to END)

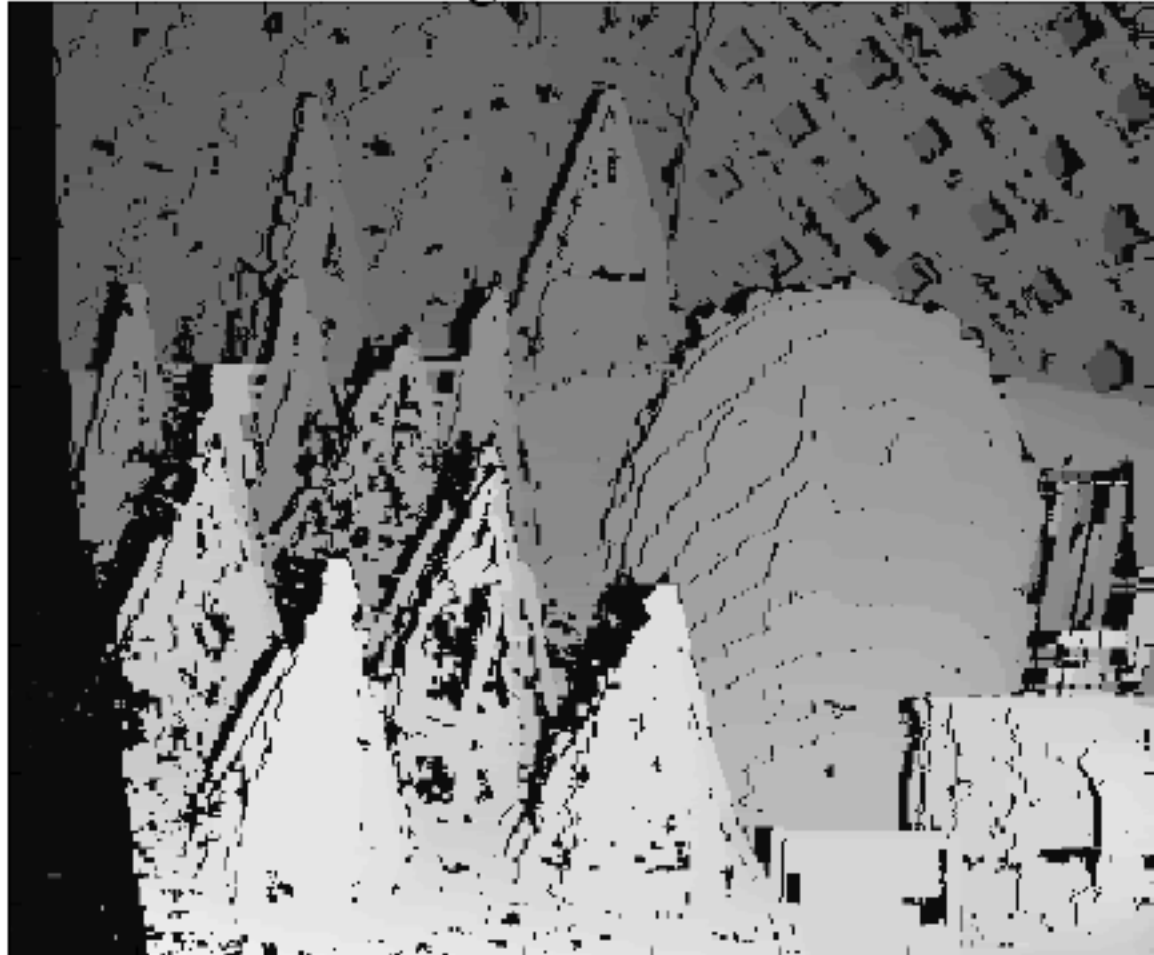


Every pixel in left column now is marked with either a disparity value, or an occlusion label.

Proceed for every scanline in left image.

Example

Result of DP alg



Result without DP (independent pixels)



Result of DP alg. Black pixels = occluded.

Siamese networks for stereo matching

Siamese Networks for Stereo Matching

- ▶ **Hand crafted features** and similarity metrics do not take into consideration all relevant geometric and radiometric invariances or occlusion patterns
- ▶ The world is too complex to specify this by hand
- ▶ Matching cost computation can be treated as **image classification problem**:

Left patch	Right patch	Label
		Good match
		Bad match
⋮	⋮	⋮

Siamese Networks for Stereo Matching

Method Overview:

- ▶ Train CNN patch-wise based on images with ground truth disparity maps (e.g., KITTI Stereo 2015: <http://www.cvlibs.net/datasets/kitti/>)

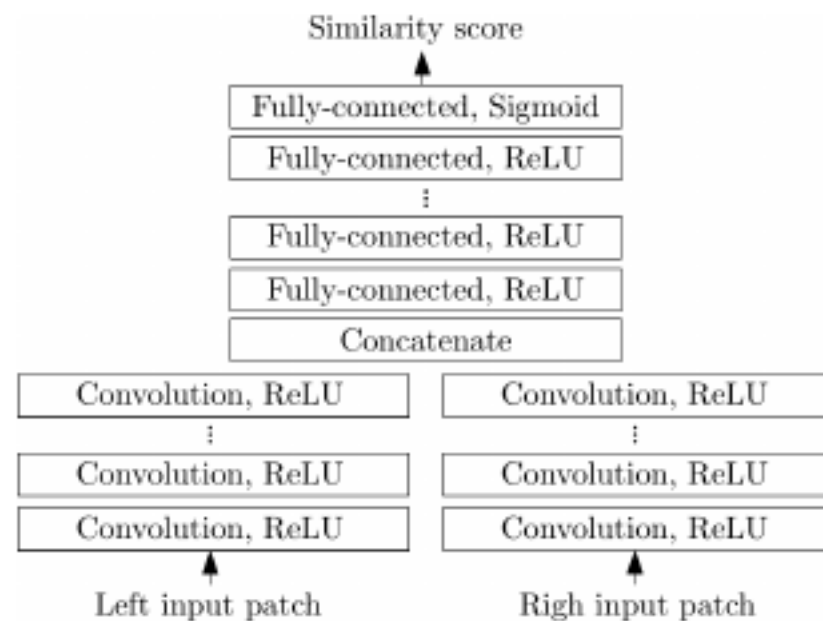


- ▶ Calculate features for each of the two images using learned model
- ▶ Correlate features between both images (dot product)
- ▶ Find maximum for every pixel (winner-takes-all) or run global optimization

Siamese Network Architecture

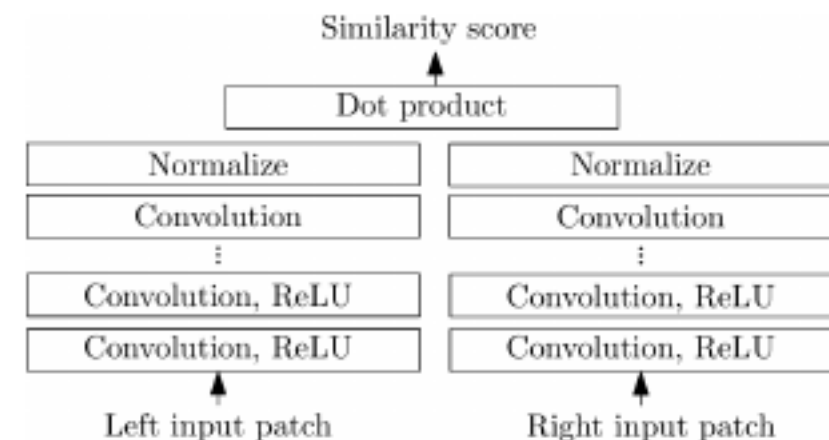
Learned Similarity:

- ▶ Learn features & sim. metric
- ▶ Potentially more expressive
- ▶ Slow (WxHxD MLP evaluations)



Cosine Similarity:

- ▶ Learn features & apply dot-product
- ▶ Features must do the heavy lifting
- ▶ Fast matching (no network eval.)



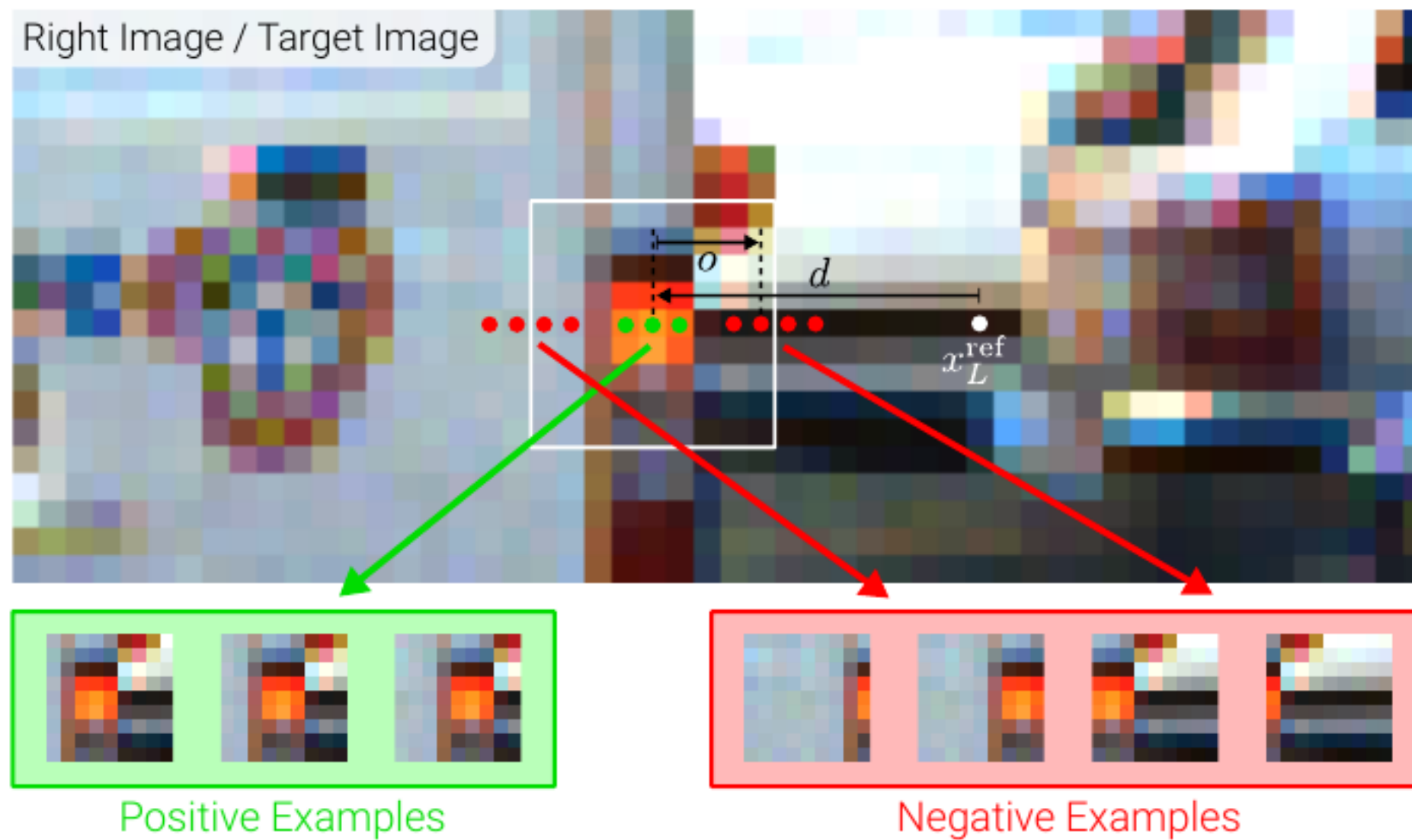
Training

The training set is composed of patch triplets:

$$\left(\mathbf{w}_L(\mathbf{x}_L^{\text{ref}}), \mathbf{w}_R(\mathbf{x}_R^{\text{neg}}), \mathbf{w}_R(\mathbf{x}_R^{\text{pos}}) \right)$$

- ▶ $\mathbf{w}_L(\mathbf{x}_L)$ is an image patch from the left image centered at $\mathbf{x}_L = (x_L, y_L)$
- ▶ $\mathbf{w}_R(\mathbf{x}_R)$ is an image patch from the right image centered at $\mathbf{x}_R = (x_R, y_R)$
- ▶ Negative example: $\mathbf{x}_R^{\text{neg}} = (x_L^{\text{ref}} - d + o_{\text{neg}}, y_L^{\text{ref}})$
 - ▶ Offsets o_{neg} drawn from $\mathcal{U}(\{-N_{\text{hi}}, \dots, -N_{\text{lo}}, N_{\text{lo}}, \dots, N_{\text{hi}}\})$
- ▶ Positive example: $\mathbf{x}_R^{\text{pos}} = (x_L^{\text{ref}} - d + o_{\text{pos}}, y_L^{\text{ref}})$
 - ▶ Offsets o_{pos} drawn from $\mathcal{U}(\{-P_{\text{hi}}, \dots, P_{\text{hi}}\})$
- ▶ Here, d denotes the true disparity for a pixel (provided as ground truth)
- ▶ Typically, $P_{\text{hi}} = 1$, $N_{\text{low}} = 3$ and $N_{\text{hi}} = 6 \Rightarrow$ hard negative mining

Training



Zbontar and LeCun: Stereo Matching by Training a Convolutional Neural Network to Compare Image Patches. JMLR, 2016.

42

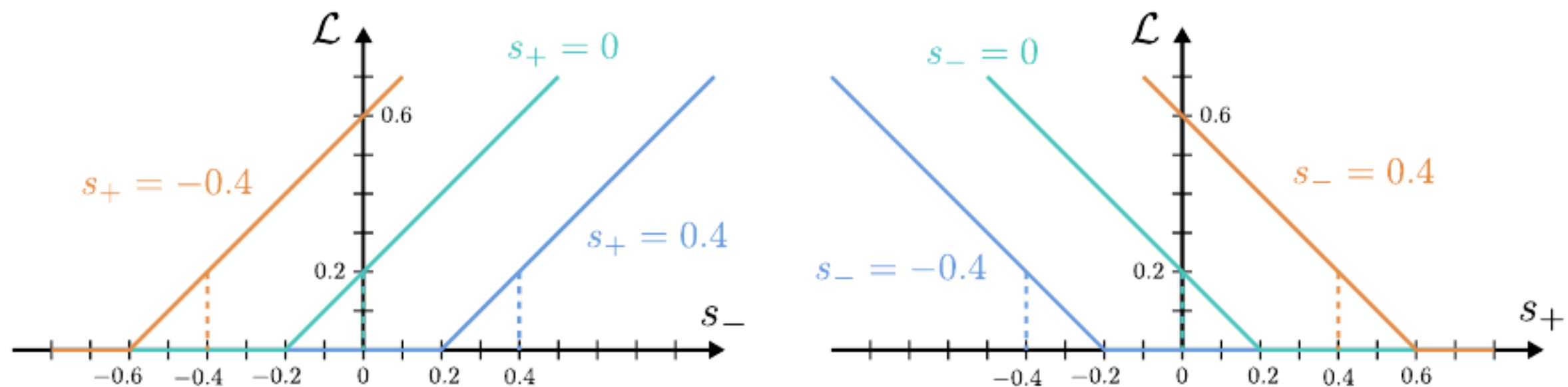
Loss Function

Hinge Loss:

$$\mathcal{L} = \max(0, m + s_- - s_+)$$

- ▶ s_- / s_+ is the score of the network for the negative/positive example
- ▶ The loss is zero when the similarity of the positive example is greater than the similarity of the negative example by at least margin m
- ▶ This prevents separation of pos/neg features that are already well separated
- ▶ Gives the model the capacity to focus on the hard cases
- ▶ The margin m is a tunable hyperparameter (the paper uses $m = 0.2$)

Loss Function

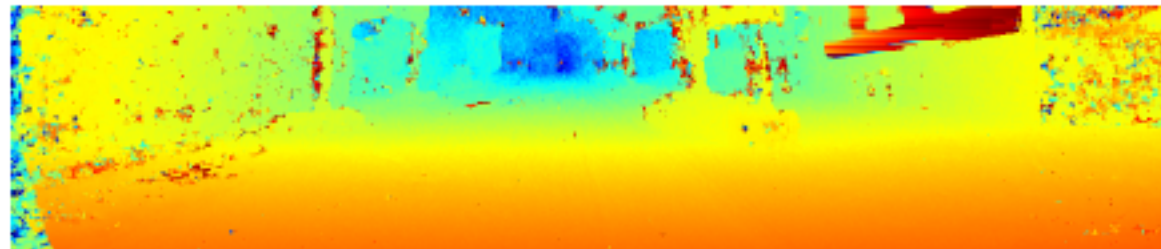


$$\mathcal{L} = \max(0, m + s_- - s_+)$$

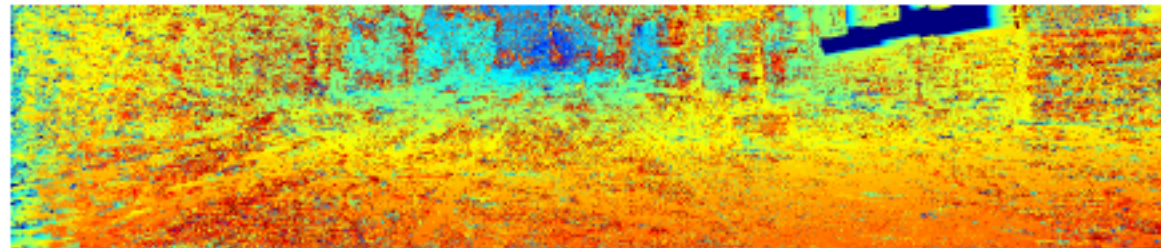
Winner-takes-All Results



Left Input Image



Siamese Network

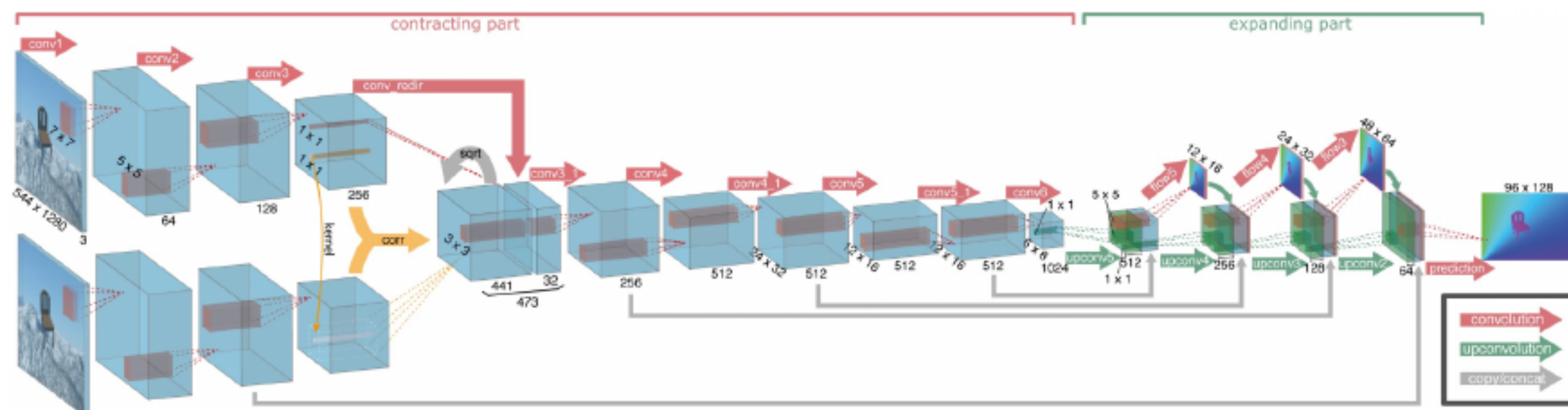


Standard Block Matching

Zbontar and LeCun: Stereo Matching by Training a Convolutional Neural Network to Compare Image Patches. JMLR, 2016.

45

DispNet

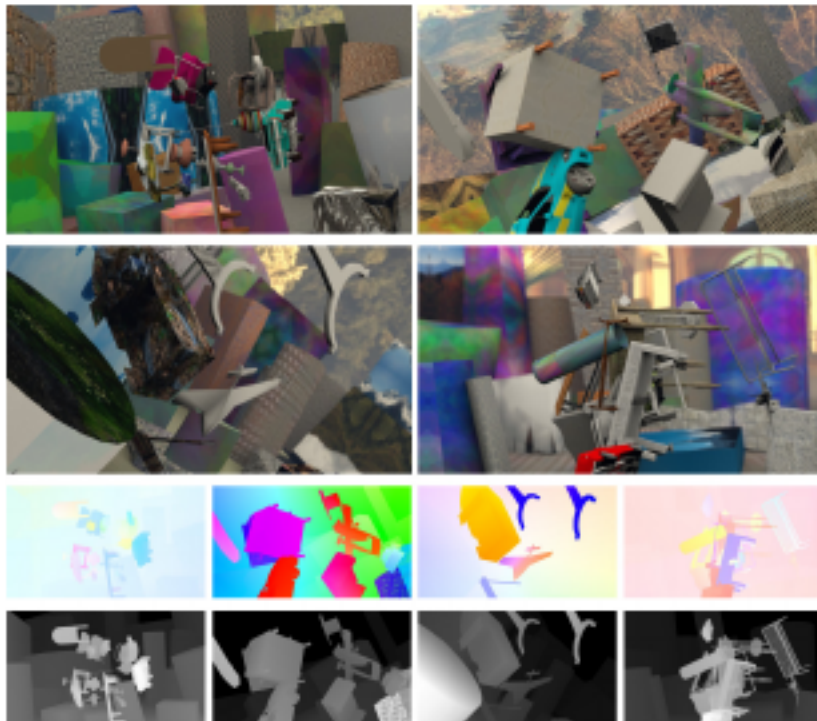


- ▶ DispNet was the first end-to-end trained deep neural network for stereo
- ▶ It used a U-Net like architecture with skip-Connections to retain details
- ▶ Correlation layer (40px displacement corresponding to 160px in input image)
- ▶ Multi-scale loss (disparity error in pixels), curriculum learning (learn easy-to-hard)

Mayer et al.: A Large Dataset to Train Convolutional Networks for Disparity, Optical Flow, and Scene Flow Estimation. CVPR, 2016.

58

Synthetic Datasets



Flying Things



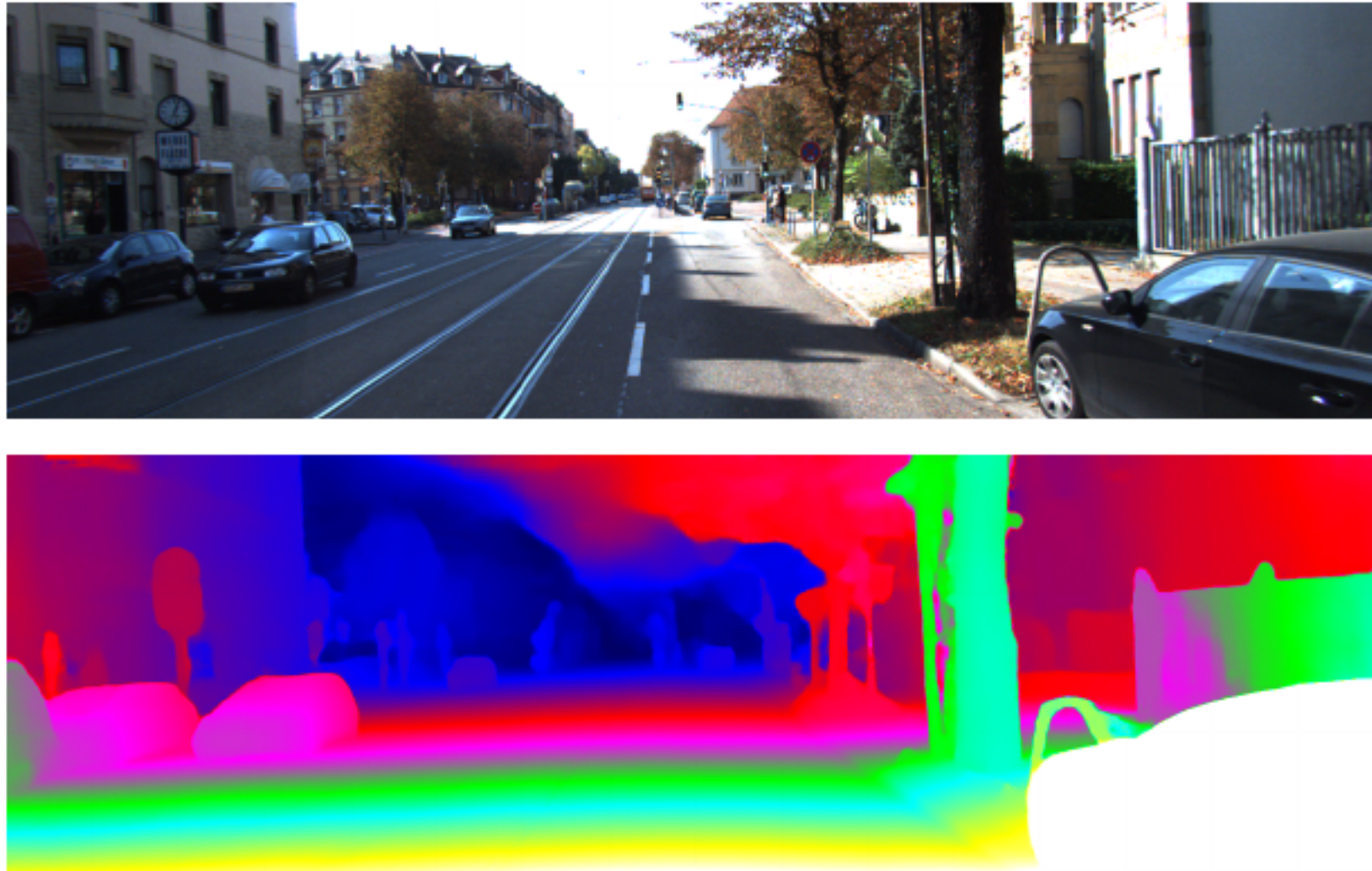
Monkaa

- ▶ Pretraining on large synthetic datasets (cheap annotations)
- ▶ Finetuning on little real data (expensive annotations)

Mayer et al.: A Large Dataset to Train Convolutional Networks for Disparity, Optical Flow, and Scene Flow Estimation. CVPR, 2016.

59

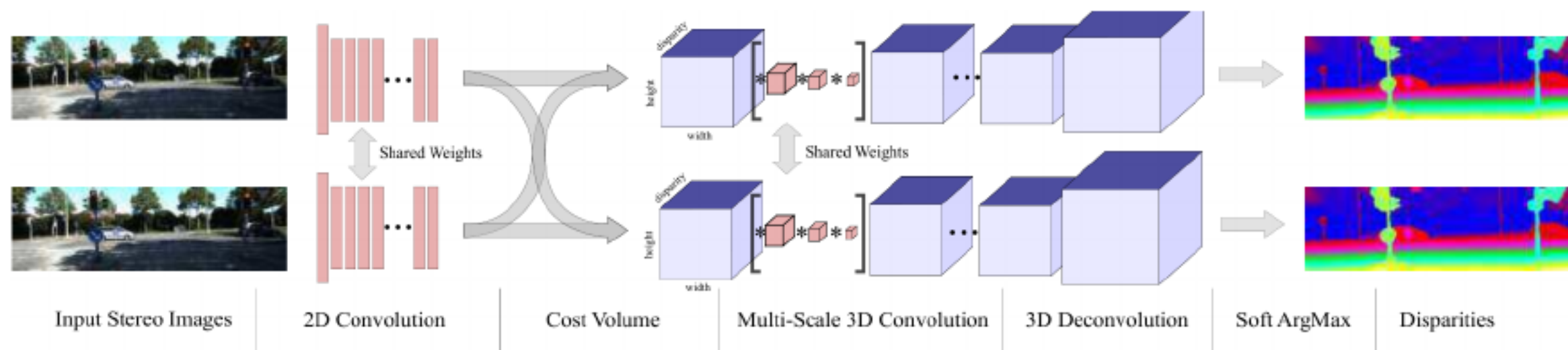
DispNet Results on KITTI Dataset



Mayer et al.: A Large Dataset to Train Convolutional Networks for Disparity, Optical Flow, and Scene Flow Estimation. CVPR, 2016.

60

GC-Net



- Key idea: calculate disparity cost volume and apply 3D convolutions on it
- Convert the learned matching cost $c_\theta(d)$ to disparity via the expectation:

$$d^* = \mathbb{E}[d] = \sum_{d=0}^D \underbrace{\text{softmax}(-c_\theta(d))}_{p(d)} \cdot d$$

- Slightly better performance but large memory requirements (3D feature volume)

Stereo Mixture Density Networks (SMD-Nets)



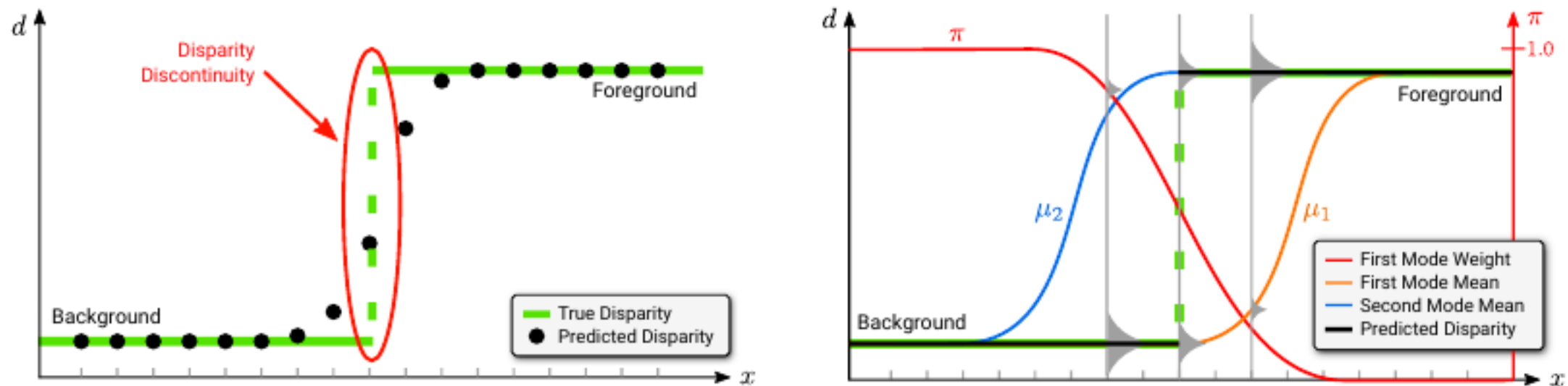
- Stereo Mixture Density Networks predict sharper boundaries at higher resolution

Stereo Mixture Density Networks (SMD-Nets)



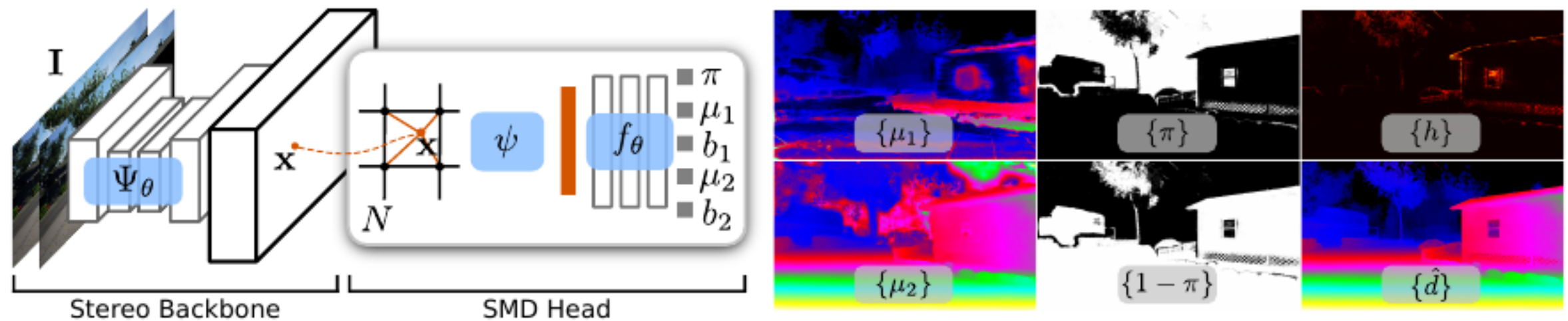
- Stereo Mixture Density Networks predict sharper boundaries at higher resolution

Stereo Mixture Density Networks (SMD-Nets)



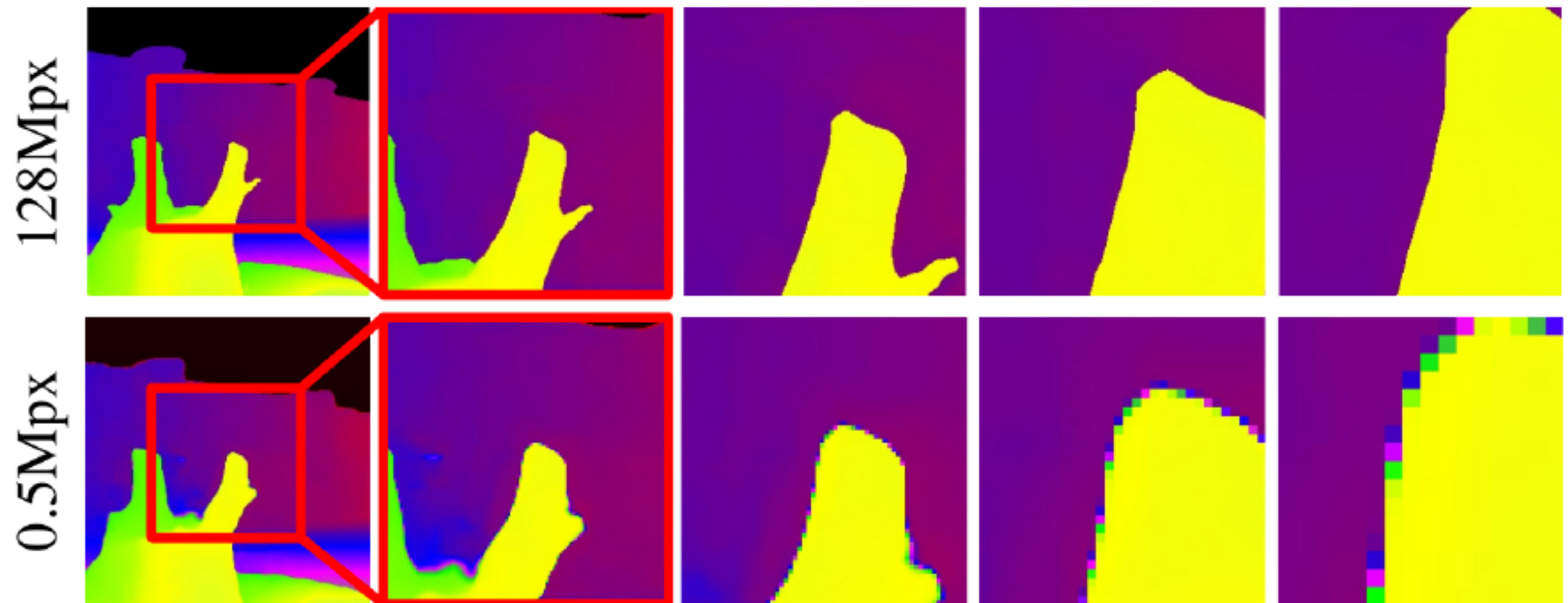
- ▶ Left: Classical deep networks for stereo regression suffer from smoothness bias and hence continuously interpolate object boundaries
- ▶ Right: SMD-Nets predict a bimodal (Laplacian) mixture distribution (gray) which allows to accurately capture uncertainty close to depth discontinuities

Stereo Mixture Density Networks (SMD-Nets)



- SMD-Nets use a 2D or 3D convolutional backbone in combination with a shallow MLP head that regresses the distribution parameters from interpolated features
- This enables training and inference at arbitrary spatial resolution

Stereo Mixture Density Networks (SMD-Nets)



- Top: SMD-Net, Bottom: Traditional stereo regression network